In [1]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from xgboost import XGBClassifier
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import LabelEncoder

from sklearn.model_selection import GridSearchCV

import warnings
warnings.filterwarnings("ignore")
```

In [2]:
```
downloaded_dataset =pd.read_csv(
    r"C:\Network Intrusion Detection System\KDDCup Data 10 Percent (1).csv",
    header=None)
print("First 10 Elements from the Dataset - KDDCup Data 10 Percent.csv:")
downloaded_dataset.head(10)
```

```
First 10 Elements from the Dataset - KDDCup Data 10 Percent.csv:
```

Out[2]:

	0	1	2	3	4	5	6	7	8	9	...	32	33	34	35	36	37	38	39	40
0	0	tcp	http	SF	181	5450	0	0	0	0	...	9	1.0	0.0	0.11	0.00	0.0	0.0	0.0	0.0
1	0	tcp	http	SF	239	486	0	0	0	0	...	19	1.0	0.0	0.05	0.00	0.0	0.0	0.0	0.0
2	0	tcp	http	SF	235	1337	0	0	0	0	...	29	1.0	0.0	0.03	0.00	0.0	0.0	0.0	0.0
3	0	tcp	http	SF	219	1337	0	0	0	0	...	39	1.0	0.0	0.03	0.00	0.0	0.0	0.0	0.0
4	0	tcp	http	SF	217	2032	0	0	0	0	...	49	1.0	0.0	0.02	0.00	0.0	0.0	0.0	0.0
5	0	tcp	http	SF	217	2032	0	0	0	0	...	59	1.0	0.0	0.02	0.00	0.0	0.0	0.0	0.0
6	0	tcp	http	SF	212	1940	0	0	0	0	...	69	1.0	0.0	1.00	0.04	0.0	0.0	0.0	0.0
7	0	tcp	http	SF	159	4087	0	0	0	0	...	79	1.0	0.0	0.09	0.04	0.0	0.0	0.0	0.0
8	0	tcp	http	SF	210	151	0	0	0	0	...	89	1.0	0.0	0.12	0.04	0.0	0.0	0.0	0.0
9	0	tcp	http	SF	212	786	0	0	0	1	...	99	1.0	0.0	0.12	0.05	0.0	0.0	0.0	0.0

10 rows × 42 columns

In [3]:
```
print("Last 10 Elements from the Dataset - KDDCup Data 10 Percent.csv:")
downloaded_dataset.tail(10)
```

Last 10 Elements from the Dataset - KDDCup Data 10 Percent.csv:

Out[3]:

	0	1	2	3	4	5	6	7	8	9	...	32	33	34	35	36	37	38
494011	0	tcp	http	SF	308	662	0	0	0	0	...	255	1.0	0.0	0.03	0.06	0.00	0.01
494012	0	tcp	http	SF	291	1862	0	0	0	0	...	255	1.0	0.0	0.02	0.05	0.00	0.01
494013	0	tcp	http	SF	289	244	0	0	0	0	...	255	1.0	0.0	0.02	0.05	0.00	0.01
494014	0	tcp	http	SF	306	662	0	0	0	0	...	255	1.0	0.0	0.02	0.05	0.00	0.01
494015	0	tcp	http	SF	289	1862	0	0	0	0	...	255	1.0	0.0	0.01	0.05	0.00	0.01
494016	0	tcp	http	SF	310	1881	0	0	0	0	...	255	1.0	0.0	0.01	0.05	0.00	0.01
494017	0	tcp	http	SF	282	2286	0	0	0	0	...	255	1.0	0.0	0.17	0.05	0.00	0.01
494018	0	tcp	http	SF	203	1200	0	0	0	0	...	255	1.0	0.0	0.06	0.05	0.06	0.01
494019	0	tcp	http	SF	291	1200	0	0	0	0	...	255	1.0	0.0	0.04	0.05	0.04	0.01
494020	0	tcp	http	SF	219	1234	0	0	0	0	...	255	1.0	0.0	0.17	0.05	0.00	0.01

10 rows × 42 columns

In [4]:

```
with open(
    "kddcup.txt", "r"
) as kddcup:  # A handle for reading the contents in the File: kddcup.txt
    # Opening the contents from the file, in order to be manipulated...
    kddcup_contents = kddcup.read()
    # I have customed/manipulated the data entry here, by slicing
    data = kddcup_contents[182:].strip("\n")
    # the contents from the starting position of 182; because I only want the colum
    # KDDCup Data 10 Percent.csv

# Now we can itereate the contents from the Dataset: KDDCup Data 10
data_scrapping = data.split("\n")
# Percent.csv, and store them in the empty List Variable: stored_features.

# for contents in data_scrapping:
#     # print(contents)   # Un-Comment this to view the contents stored in the List
# stored_features...
# ------------------------------------------------- NOTE -------------------------
# So we know the contents in the List Variable: data_scrapping, is an element of a
#
# List Variable: extracted_data - By observance, we know that each Key is the Colum
#                                 its corresponding connection.This allows us to se
#                                 elements are seperated as its singular value.
#
# List Variable: extracted_features - Stores the data entry which will be used for
#                                     Percent.csv Column Names
#
# List Variable: extracted_connections - Stores the data entry which will be used f
#                                        10 Percent.csv connection type.
extracted_data = [
    word for lists_data in data_scrapping for word in lists_data.split(": ")
```

```
]
extracted_features = [
    contents for i, contents in enumerate(extracted_data) if i % 2 == 0
]
extracted_connections = [
    contents for i, contents in enumerate(extracted_data) if i % 2 != 0
]

extracted_features.append("connection")

print("Extracted Features:   ", extracted_features)  # Displays Column Name
# Displays Connection Type
print("Extracted Connections:", extracted_connections)
```

```
Extracted Features:    ['duration', 'protocol_type', 'service', 'flag', 'src_bytes',
'dst_bytes', 'land', 'wrong_fragment', 'urgent', 'hot', 'num_failed_logins', 'logged
_in', 'num_compromised', 'root_shell', 'su_attempted', 'num_root', 'num_file_creatio
ns', 'num_shells', 'num_access_files', 'num_outbound_cmds', 'is_host_login', 'is_gue
st_login', 'count', 'srv_count', 'serror_rate', 'srv_serror_rate', 'rerror_rate', 's
rv_rerror_rate', 'same_srv_rate', 'diff_srv_rate', 'srv_diff_host_rate', 'dst_host_c
ount', 'dst_host_srv_count', 'dst_host_same_srv_rate', 'dst_host_diff_srv_rate', 'ds
t_host_same_src_port_rate', 'dst_host_srv_diff_host_rate', 'dst_host_serror_rate',
'dst_host_srv_serror_rate', 'dst_host_rerror_rate', 'dst_host_srv_rerror_rate', 'con
nection']
Extracted Connections: ['continuous.', 'symbolic.', 'symbolic.', 'symbolic.', 'conti
nuous.', 'continuous.', 'symbolic.', 'continuous.', 'continuous.', 'continuous.', 'c
ontinuous.', 'symbolic.', 'continuous.', 'continuous.', 'continuous.', 'continuou
s.', 'continuous.', 'continuous.', 'continuous.', 'continuous.', 'symbolic.', 'symbo
lic.', 'continuous.', 'continuous.', 'continuous.', 'continuous.', 'continuous.', 'c
ontinuous.', 'continuous.', 'continuous.', 'continuous.', 'continuous.', 'continuou
s.', 'continuous.', 'continuous.', 'continuous.', 'continuous.', 'continuous.', 'con
tinuous.', 'continuous.', 'continuous.']
```

In [5]:
```
# This will put the column names in sequence of the List Variable: extracted_featur
downloaded_dataset.columns = extracted_features

# Saved in local directory:
downloaded_dataset.to_csv("KDDCup Data 10 Percent (Edited).csv")

# This becomes the dataset for current use:
edited_dataset = pd.read_csv("KDDCup Data 10 Percent (Edited).csv")
```

In [6]:
```
# NOTE: This line of code is used for debugging. When we print the head of the data
# dataset.head()  # Column: Unnamed: 0...

# So the column is dropped in this line of code:
dropped_feature = edited_dataset.drop(columns="Unnamed: 0", axis=1, inplace=True)
```

In [7]:
```
print("First 10 Elements from the Dataset - KDDCup Data 10 Percent (Edited).csv:")
edited_dataset[:10]
```

```
First 10 Elements from the Dataset - KDDCup Data 10 Percent (Edited).csv:
```

Out[7]:

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urg
0	0	tcp	http	SF	181	5450	0	0	
1	0	tcp	http	SF	239	486	0	0	
2	0	tcp	http	SF	235	1337	0	0	
3	0	tcp	http	SF	219	1337	0	0	
4	0	tcp	http	SF	217	2032	0	0	
5	0	tcp	http	SF	217	2032	0	0	
6	0	tcp	http	SF	212	1940	0	0	
7	0	tcp	http	SF	159	4087	0	0	
8	0	tcp	http	SF	210	151	0	0	
9	0	tcp	http	SF	212	786	0	0	

10 rows × 42 columns

In [8]:
```
print("Last 10 Elements from the Dataset - KDDCup Data 10 Percent (Edited).csv:")
edited_dataset[-10:]
```

```
Last 10 Elements from the Dataset - KDDCup Data 10 Percent (Edited).csv:
```

Out[8]:

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment
494011	0	tcp	http	SF	308	662	0	0
494012	0	tcp	http	SF	291	1862	0	0
494013	0	tcp	http	SF	289	244	0	0
494014	0	tcp	http	SF	306	662	0	0
494015	0	tcp	http	SF	289	1862	0	0
494016	0	tcp	http	SF	310	1881	0	0
494017	0	tcp	http	SF	282	2286	0	0
494018	0	tcp	http	SF	203	1200	0	0
494019	0	tcp	http	SF	291	1200	0	0
494020	0	tcp	http	SF	219	1234	0	0

10 rows × 42 columns

The code below will list out all the column names.

In [9]:
```
print(list(edited_dataset))
```

```
['duration', 'protocol_type', 'service', 'flag', 'src_bytes', 'dst_bytes', 'land',
'wrong_fragment', 'urgent', 'hot', 'num_failed_logins', 'logged_in', 'num_compromise
d', 'root_shell', 'su_attempted', 'num_root', 'num_file_creations', 'num_shells', 'n
um_access_files', 'num_outbound_cmds', 'is_host_login', 'is_guest_login', 'count',
'srv_count', 'serror_rate', 'srv_serror_rate', 'rerror_rate', 'srv_rerror_rate', 'sa
me_srv_rate', 'diff_srv_rate', 'srv_diff_host_rate', 'dst_host_count', 'dst_host_srv
_count', 'dst_host_same_srv_rate', 'dst_host_diff_srv_rate', 'dst_host_same_src_port
_rate', 'dst_host_srv_diff_host_rate', 'dst_host_serror_rate', 'dst_host_srv_serror_
rate', 'dst_host_rerror_rate', 'dst_host_srv_rerror_rate', 'connection']
```

In [10]:
```
edited_dataset["connection"].value_counts()
```

Out[10]:
```
connection
smurf.              280790
neptune.            107201
normal.              97278
back.                 2203
satan.                1589
ipsweep.              1247
portsweep.            1040
warezclient.          1020
teardrop.              979
pod.                   264
nmap.                  231
guess_passwd.           53
buffer_overflow.        30
land.                   21
warezmaster.            20
imap.                   12
rootkit.                10
loadmodule.              9
ftp_write.               8
multihop.                7
phf.                     4
perl.                    3
spy.                     2
Name: count, dtype: int64
```

The cell abolve displays counts of each type of connections.

Let's store all the records in a dataframe normal where connection is 'normal'.

In [11]:
```
normal = edited_dataset[edited_dataset.connection == "normal."]
normal.head()
```

Out[11]:

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urg
0	0	tcp	http	SF	181	5450	0	0	
1	0	tcp	http	SF	239	486	0	0	
2	0	tcp	http	SF	235	1337	0	0	
3	0	tcp	http	SF	219	1337	0	0	
4	0	tcp	http	SF	217	2032	0	0	

5 rows × 42 columns

In [12]:
```
normal_rows = normal.shape[0]
normal_columns = normal.shape[1]
edited_dataset_rows = edited_dataset.shape[0]
edited_dataset_columns = edited_dataset.shape[1]

print(
    f"The Dataset - normal has {normal_rows:,} rows and {normal_columns} columns."
    + "\n"
    + f"The Dataset - edited_dataset has {edited_dataset_rows:,} rows and {edited_d
)
```

```
The Dataset - normal has 97,278 rows and 42 columns.
The Dataset - edited_dataset has 494,021 rows and 42 columns.
```

Prepare your personalised dataset for this task. Replace '26577' with the last 5 digits of your student number. If it starts with zero, please use 6 as the first digit.

Save the data in to a csv file using the codes in the cells below.

In [13]:
```
rows = np.random.choice(edited_dataset.index.values, 24861)
dataset = edited_dataset.loc[rows]

dataset.to_csv("Latest.csv", index=False)
```

Replace name and number with your name and student number. Write it to a csv file. You are required to use this dataset to complete this task.

In [14]:
```
dataset = pd.read_csv("Latest.csv")

print("First 10 Elements from the Dataset - KDDCup Data 10 Percent (Edited).csv:")
dataset[:10]
```

```
First 10 Elements from the Dataset - KDDCup Data 10 Percent (Edited).csv:
```

Out[14]:

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urg
0	0	tcp	http	SF	341	16123	0	0	
1	2638	udp	other	SF	145	105	0	0	
2	0	icmp	ecr_i	SF	1032	0	0	0	
3	0	tcp	private	REJ	0	0	0	0	
4	0	icmp	ecr_i	SF	1032	0	0	0	
5	0	tcp	private	S0	0	0	0	0	
6	0	tcp	http	SF	320	334	0	0	
7	0	icmp	ecr_i	SF	520	0	0	0	
8	0	icmp	ecr_i	SF	1032	0	0	0	
9	0	tcp	smtp	SF	1105	328	0	0	

10 rows × 42 columns

In [15]:
```
print("Last 10 Elements from the Dataset - KDDCup Data 10 Percent (Edited).csv:")
dataset[-10:]
```

```
Last 10 Elements from the Dataset - KDDCup Data 10 Percent (Edited).csv:
```

Out[15]:

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment
24851	0	tcp	http	SF	345	398	0	0
24852	0	tcp	http	SF	343	6177	0	0
24853	0	icmp	ecr_i	SF	1032	0	0	0
24854	0	icmp	ecr_i	SF	1032	0	0	0
24855	0	icmp	ecr_i	SF	1032	0	0	0
24856	0	tcp	private	S0	0	0	0	0
24857	0	tcp	private	S0	0	0	0	0
24858	4	tcp	smtp	SF	699	328	0	0
24859	0	icmp	ecr_i	SF	1032	0	0	0
24860	0	icmp	ecr_i	SF	1032	0	0	0

10 rows × 42 columns

In [16]:
```
dataset["connection"].value_counts()
```

Out[16]:
```
connection
smurf.              14066
neptune.             5429
normal.              4905
back.                 110
ipsweep.               76
satan.                 64
warezclient.           62
portsweep.             57
teardrop.              55
pod.                   14
nmap.                  12
guess_passwd.           5
buffer_overflow.        3
loadmodule.             2
imap.                   1
Name: count, dtype: int64
```

In [17]:
```
dataset.head()
```

Out[17]:

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urg
0	0	tcp	http	SF	341	16123	0	0	
1	2638	udp	other	SF	145	105	0	0	
2	0	icmp	ecr_i	SF	1032	0	0	0	
3	0	tcp	private	REJ	0	0	0	0	
4	0	icmp	ecr_i	SF	1032	0	0	0	

5 rows × 42 columns

Data Preprocessing

In [18]:
```
# will return the data types of all the columns
dataset.dtypes
```

Out[18]:
```
duration                          int64
protocol_type                    object
service                          object
flag                             object
src_bytes                         int64
dst_bytes                         int64
land                              int64
wrong_fragment                    int64
urgent                            int64
hot                               int64
num_failed_logins                 int64
logged_in                         int64
num_compromised                   int64
root_shell                        int64
su_attempted                      int64
num_root                          int64
num_file_creations                int64
num_shells                        int64
num_access_files                  int64
num_outbound_cmds                 int64
is_host_login                     int64
is_guest_login                    int64
count                             int64
srv_count                         int64
serror_rate                     float64
srv_serror_rate                 float64
rerror_rate                     float64
srv_rerror_rate                 float64
same_srv_rate                   float64
diff_srv_rate                   float64
srv_diff_host_rate              float64
dst_host_count                    int64
dst_host_srv_count                int64
dst_host_same_srv_rate          float64
dst_host_diff_srv_rate          float64
dst_host_same_src_port_rate     float64
dst_host_srv_diff_host_rate     float64
dst_host_serror_rate            float64
dst_host_srv_serror_rate        float64
dst_host_rerror_rate            float64
dst_host_srv_rerror_rate        float64
connection                       object
dtype: object
```

In [19]:
```
dataset.shape
```

Out[19]:
```
(24861, 42)
```

In [20]:
```
# will generate descriptive statistics of your dataset
dataset.describe()
```

Out[20]:

	duration	src_bytes	dst_bytes	land	wrong_fragment	urgent	
count	24861.000000	2.486100e+04	24861.000000	24861.0	24861.000000	24861.0	2486
mean	58.158039	1.878291e+03	606.907244	0.0	0.006999	0.0	
std	794.623408	6.811794e+04	4060.136078	0.0	0.140506	0.0	
min	0.000000	0.000000e+00	0.000000	0.0	0.000000	0.0	
25%	0.000000	4.300000e+01	0.000000	0.0	0.000000	0.0	
50%	0.000000	5.200000e+02	0.000000	0.0	0.000000	0.0	
75%	0.000000	1.032000e+03	0.000000	0.0	0.000000	0.0	
max	31061.000000	5.135678e+06	288690.000000	0.0	3.000000	0.0	3

8 rows × 38 columns

In [21]:
```
dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 24861 entries, 0 to 24860
Data columns (total 42 columns):
 #   Column                       Non-Null Count  Dtype  
---  ------                       --------------  -----  
 0   duration                     24861 non-null  int64  
 1   protocol_type                24861 non-null  object 
 2   service                      24861 non-null  object 
 3   flag                         24861 non-null  object 
 4   src_bytes                    24861 non-null  int64  
 5   dst_bytes                    24861 non-null  int64  
 6   land                         24861 non-null  int64  
 7   wrong_fragment               24861 non-null  int64  
 8   urgent                       24861 non-null  int64  
 9   hot                          24861 non-null  int64  
 10  num_failed_logins            24861 non-null  int64  
 11  logged_in                    24861 non-null  int64  
 12  num_compromised              24861 non-null  int64  
 13  root_shell                   24861 non-null  int64  
 14  su_attempted                 24861 non-null  int64  
 15  num_root                     24861 non-null  int64  
 16  num_file_creations           24861 non-null  int64  
 17  num_shells                   24861 non-null  int64  
 18  num_access_files             24861 non-null  int64  
 19  num_outbound_cmds            24861 non-null  int64  
 20  is_host_login                24861 non-null  int64  
 21  is_guest_login               24861 non-null  int64  
 22  count                        24861 non-null  int64  
 23  srv_count                    24861 non-null  int64  
 24  serror_rate                  24861 non-null  float64
 25  srv_serror_rate              24861 non-null  float64
 26  rerror_rate                  24861 non-null  float64
 27  srv_rerror_rate              24861 non-null  float64
 28  same_srv_rate                24861 non-null  float64
 29  diff_srv_rate                24861 non-null  float64
 30  srv_diff_host_rate           24861 non-null  float64
 31  dst_host_count               24861 non-null  int64  
 32  dst_host_srv_count           24861 non-null  int64  
 33  dst_host_same_srv_rate       24861 non-null  float64
 34  dst_host_diff_srv_rate       24861 non-null  float64
 35  dst_host_same_src_port_rate  24861 non-null  float64
 36  dst_host_srv_diff_host_rate  24861 non-null  float64
 37  dst_host_serror_rate         24861 non-null  float64
 38  dst_host_srv_serror_rate     24861 non-null  float64
 39  dst_host_rerror_rate         24861 non-null  float64
 40  dst_host_srv_rerror_rate     24861 non-null  float64
 41  connection                   24861 non-null  object 
dtypes: float64(15), int64(23), object(4)
memory usage: 8.0+ MB
```

In [22]:
```
# Tells how many missing values are present in each column
dataset.isnull().sum()
```

Out[22]:
```
duration                        0
protocol_type                   0
service                         0
flag                            0
src_bytes                       0
dst_bytes                       0
land                            0
wrong_fragment                  0
urgent                          0
hot                             0
num_failed_logins               0
logged_in                       0
num_compromised                 0
root_shell                      0
su_attempted                    0
num_root                        0
num_file_creations              0
num_shells                      0
num_access_files                0
num_outbound_cmds               0
is_host_login                   0
is_guest_login                  0
count                           0
srv_count                       0
serror_rate                     0
srv_serror_rate                 0
rerror_rate                     0
srv_rerror_rate                 0
same_srv_rate                   0
diff_srv_rate                   0
srv_diff_host_rate              0
dst_host_count                  0
dst_host_srv_count              0
dst_host_same_srv_rate          0
dst_host_diff_srv_rate          0
dst_host_same_src_port_rate     0
dst_host_srv_diff_host_rate     0
dst_host_serror_rate            0
dst_host_srv_serror_rate        0
dst_host_rerror_rate            0
dst_host_srv_rerror_rate        0
connection                      0
dtype: int64
```

In [23]:
```
dataset.drop_duplicates(inplace=True)
dataset.shape
```

Out[23]:
```
(9807, 42)
```

In [24]:
```
dataset.columns
```

Out[24]:
```
Index(['duration', 'protocol_type', 'service', 'flag', 'src_bytes',
       'dst_bytes', 'land', 'wrong_fragment', 'urgent', 'hot',
       'num_failed_logins', 'logged_in', 'num_compromised', 'root_shell',
       'su_attempted', 'num_root', 'num_file_creations', 'num_shells',
       'num_access_files', 'num_outbound_cmds', 'is_host_login',
       'is_guest_login', 'count', 'srv_count', 'serror_rate',
       'srv_serror_rate', 'rerror_rate', 'srv_rerror_rate', 'same_srv_rate',
       'diff_srv_rate', 'srv_diff_host_rate', 'dst_host_count',
       'dst_host_srv_count', 'dst_host_same_srv_rate',
       'dst_host_diff_srv_rate', 'dst_host_same_src_port_rate',
       'dst_host_srv_diff_host_rate', 'dst_host_serror_rate',
       'dst_host_srv_serror_rate', 'dst_host_rerror_rate',
       'dst_host_srv_rerror_rate', 'connection'],
      dtype='object')
```

In [25]:
```
# This code is used to find all categorical (string/object) columns in a Pandas Dat

obj_cols = dataset.select_dtypes(include=["object"]).columns
print(obj_cols)
```

```
Index(['protocol_type', 'service', 'flag', 'connection'], dtype='object')
```

In [26]:
```
#This code is used to convert categorical (text) columns into numerical values so t


le = LabelEncoder()
for i in obj_cols:
    dataset[i] = le.fit_transform(dataset[i])
```

In [27]:
```
dataset.dtypes
```

Out[27]:
```
duration                        int64
protocol_type                   int64
service                         int64
flag                            int64
src_bytes                       int64
dst_bytes                       int64
land                            int64
wrong_fragment                  int64
urgent                          int64
hot                             int64
num_failed_logins               int64
logged_in                       int64
num_compromised                 int64
root_shell                      int64
su_attempted                    int64
num_root                        int64
num_file_creations              int64
num_shells                      int64
num_access_files                int64
num_outbound_cmds               int64
is_host_login                   int64
is_guest_login                  int64
count                           int64
srv_count                       int64
serror_rate                   float64
srv_serror_rate               float64
rerror_rate                   float64
srv_rerror_rate               float64
same_srv_rate                 float64
diff_srv_rate                 float64
srv_diff_host_rate            float64
dst_host_count                  int64
dst_host_srv_count              int64
dst_host_same_srv_rate        float64
dst_host_diff_srv_rate        float64
dst_host_same_src_port_rate   float64
dst_host_srv_diff_host_rate   float64
dst_host_serror_rate          float64
dst_host_srv_serror_rate      float64
dst_host_rerror_rate          float64
dst_host_srv_rerror_rate      float64
connection                      int64
dtype: object
```

In [28]:
```
#This code is used to get all column names except the last column.

columns = list(dataset)
columns = columns[:-1]
```

In [29]:
```
#This code is used to standardize (scale) the numerical features before training a
# StandardScaler standardizes numerical features so that each feature has a mean of

scaler = StandardScaler()
sc_data = scaler.fit_transform(dataset[columns])
print(sc_data)
```

```
[[-0.11174437 -0.1907899  -1.01381963 ... -0.79526687 -0.39813978
  -0.39389014]
 [ 2.00597162  2.82707544  0.45846906 ... -0.79526687 -0.39813978
  -0.39389014]
 [-0.11174437 -3.20865523 -1.66817015 ... -0.79526687 -0.39813978
  -0.39389014]
 ...
 [-0.11174437 -0.1907899  -1.01381963 ... -0.79526687 -0.36826902
  -0.3639093 ]
 [-0.11174437 -0.1907899   0.78564432 ...  1.26033027 -0.39813978
  -0.39389014]
 [-0.10853328 -0.1907899   1.11281959 ... -0.79526687 -0.39813978
  -0.39389014]]
```

In [30]:
```
# TRANSFORMED DATASET
transformed_data = pd.DataFrame(sc_data, columns=columns)
transformed_data.head()
```

Out[30]:

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragm
0	-0.111744	-0.190790	-1.013820	0.882304	-0.026956	2.319289	0.0	-0.077
1	2.005972	2.827075	0.458469	0.882304	-0.028764	-0.217609	0.0	-0.077
2	-0.111744	-3.208655	-1.668170	0.882304	-0.020582	-0.234239	0.0	-0.077
3	-0.111744	-0.190790	0.785644	-2.102140	-0.030102	-0.234239	0.0	-0.077
4	-0.111744	-0.190790	0.785644	-0.396743	-0.030102	-0.234239	0.0	-0.077

5 rows × 41 columns

In [31]:
```
full = pd.DataFrame(transformed_data)
full
```

Out[31]:

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fra
0	-0.111744	-0.190790	-1.013820	0.882304	-0.026956	2.319289	0.0	-0.
1	2.005972	2.827075	0.458469	0.882304	-0.028764	-0.217609	0.0	-0.
2	-0.111744	-3.208655	-1.668170	0.882304	-0.020582	-0.234239	0.0	-0.
3	-0.111744	-0.190790	0.785644	-2.102140	-0.030102	-0.234239	0.0	-0.
4	-0.111744	-0.190790	0.785644	-0.396743	-0.030102	-0.234239	0.0	-0.
...	...	...	...	...	...	...	...	
9802	-0.111744	-0.190790	0.785644	-0.396743	-0.030102	-0.234239	0.0	-0.
9803	-0.111744	-0.190790	-1.013820	0.882304	-0.026919	-0.171204	0.0	-0.
9804	-0.111744	-0.190790	-1.013820	0.882304	-0.026938	0.744062	0.0	-0.
9805	-0.111744	-0.190790	0.785644	-0.396743	-0.030102	-0.234239	0.0	-0.
9806	-0.108533	-0.190790	1.112820	0.882304	-0.023654	-0.182291	0.0	-0.

9807 rows × 41 columns

In [32]:
```
#This creates a new column called connection inside full.
full["connection"] = dataset["connection"]
full.head()
```

Out[32]:

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragm
0	-0.111744	-0.190790	-1.013820	0.882304	-0.026956	2.319289	0.0	-0.077
1	2.005972	2.827075	0.458469	0.882304	-0.028764	-0.217609	0.0	-0.077
2	-0.111744	-3.208655	-1.668170	0.882304	-0.020582	-0.234239	0.0	-0.077
3	-0.111744	-0.190790	0.785644	-2.102140	-0.030102	-0.234239	0.0	-0.077
4	-0.111744	-0.190790	0.785644	-0.396743	-0.030102	-0.234239	0.0	-0.077

5 rows × 42 columns

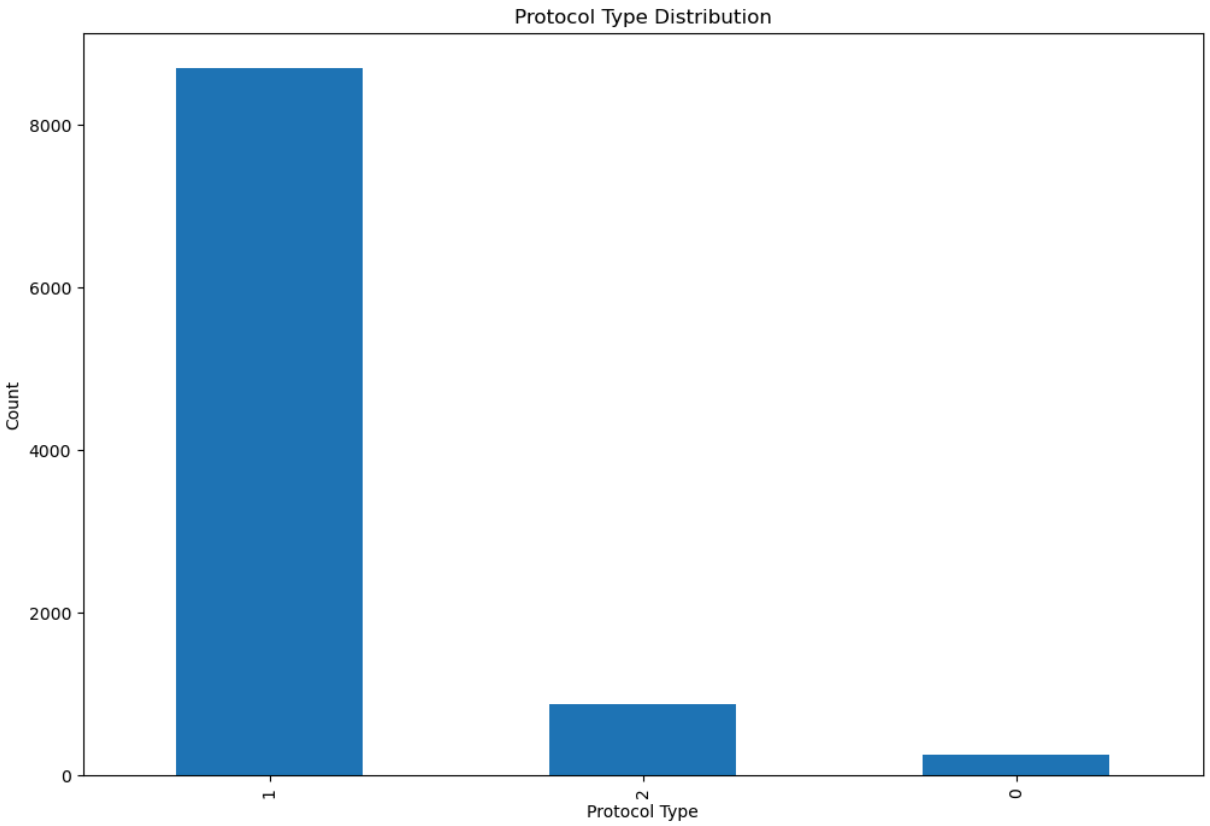
In [33]:
```
# connection → Target (Dependent variable)
target = dataset["connection"]
target.head(10)
```

Out[33]:
```
0      8
1      8
2     12
3      6
5      6
6      8
7     12
8     12
9      8
10    12
Name: connection, dtype: int64
```

Insights about the Data

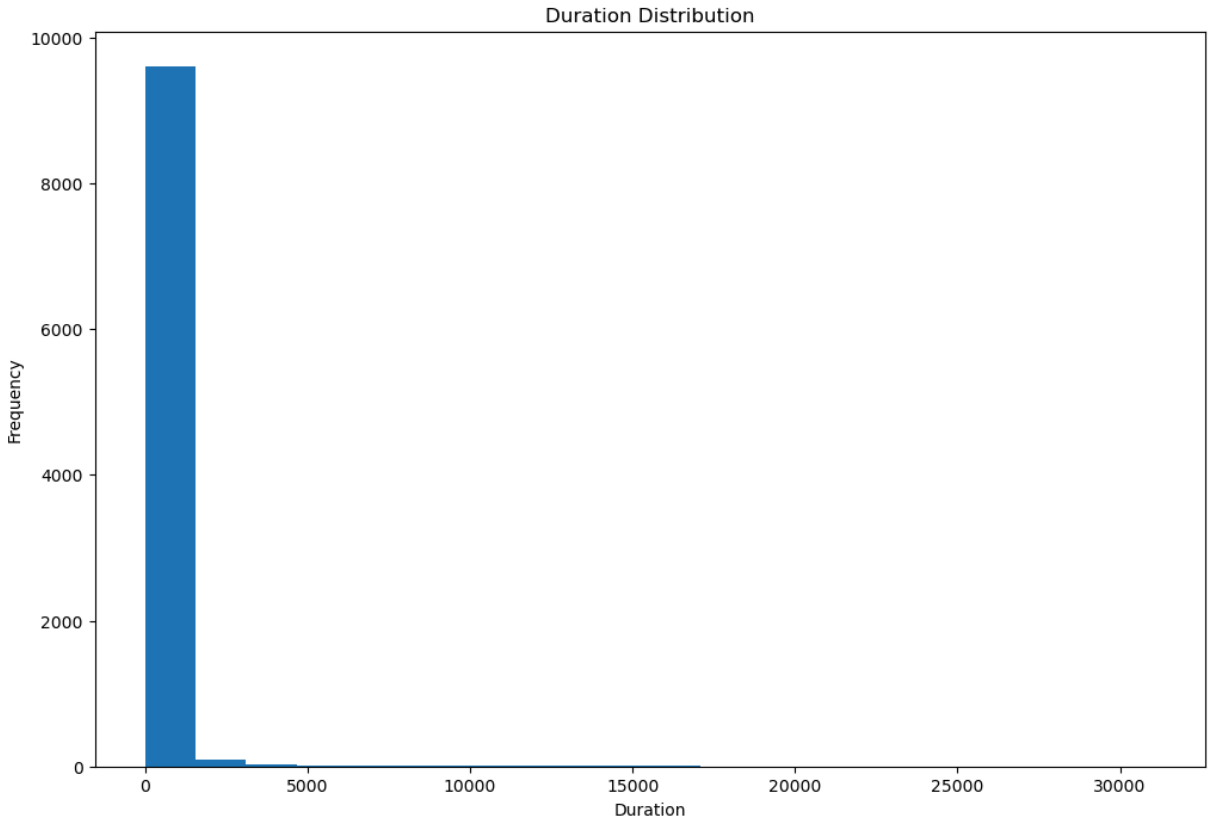
In [34]:
```
# rcParams stands for Runtime Configuration Parameters.
# It is a dictionary that stores Matplotlib's default settings, such as:
plt.rcParams["figure.figsize"] = (12, 8)
```

In [35]:
```
dataset['protocol_type'].value_counts().plot(kind='bar')
plt.xlabel('Protocol Type')
plt.ylabel('Count')
plt.title('Protocol Type Distribution')
plt.show()
```

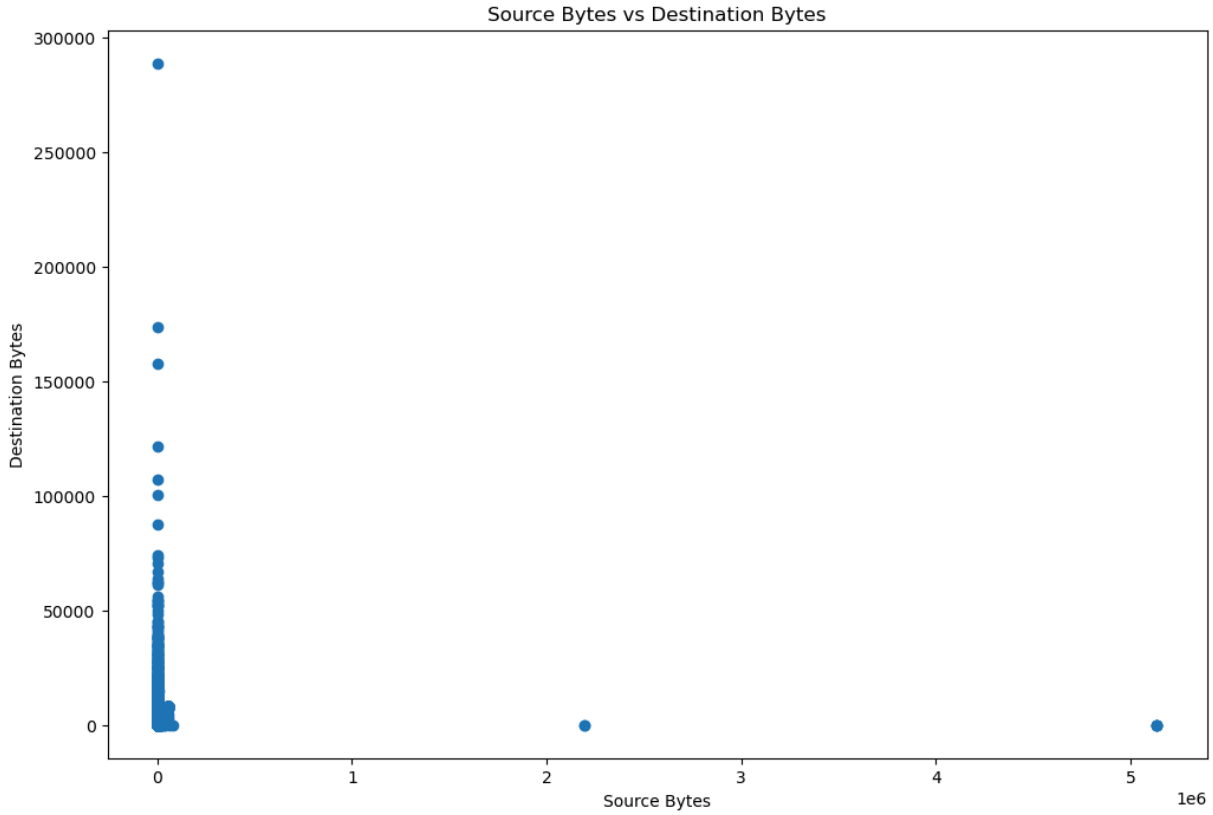


In [36]:
```
dataset['duration'].plot(kind='hist', bins=20)
plt.xlabel('Duration')
plt.ylabel('Frequency')
```

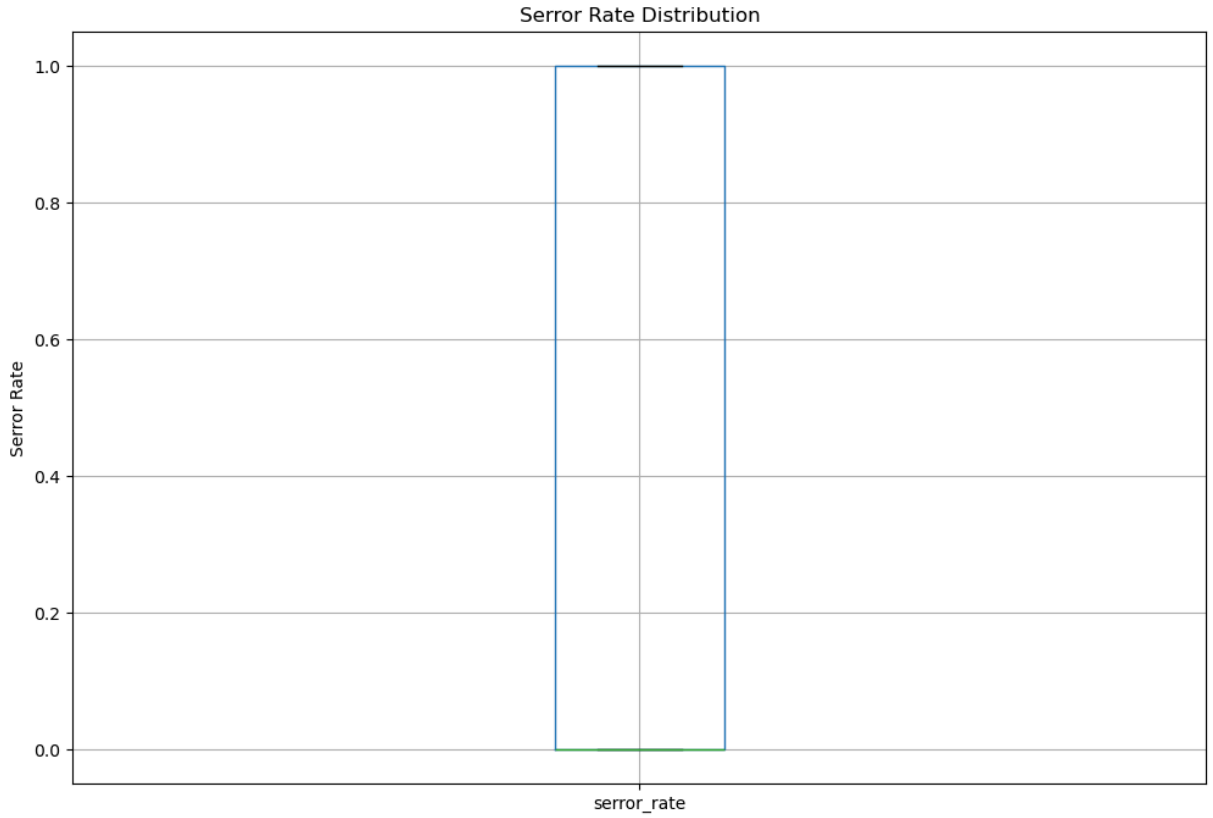
```
plt.title('Duration Distribution')
plt.show()
```



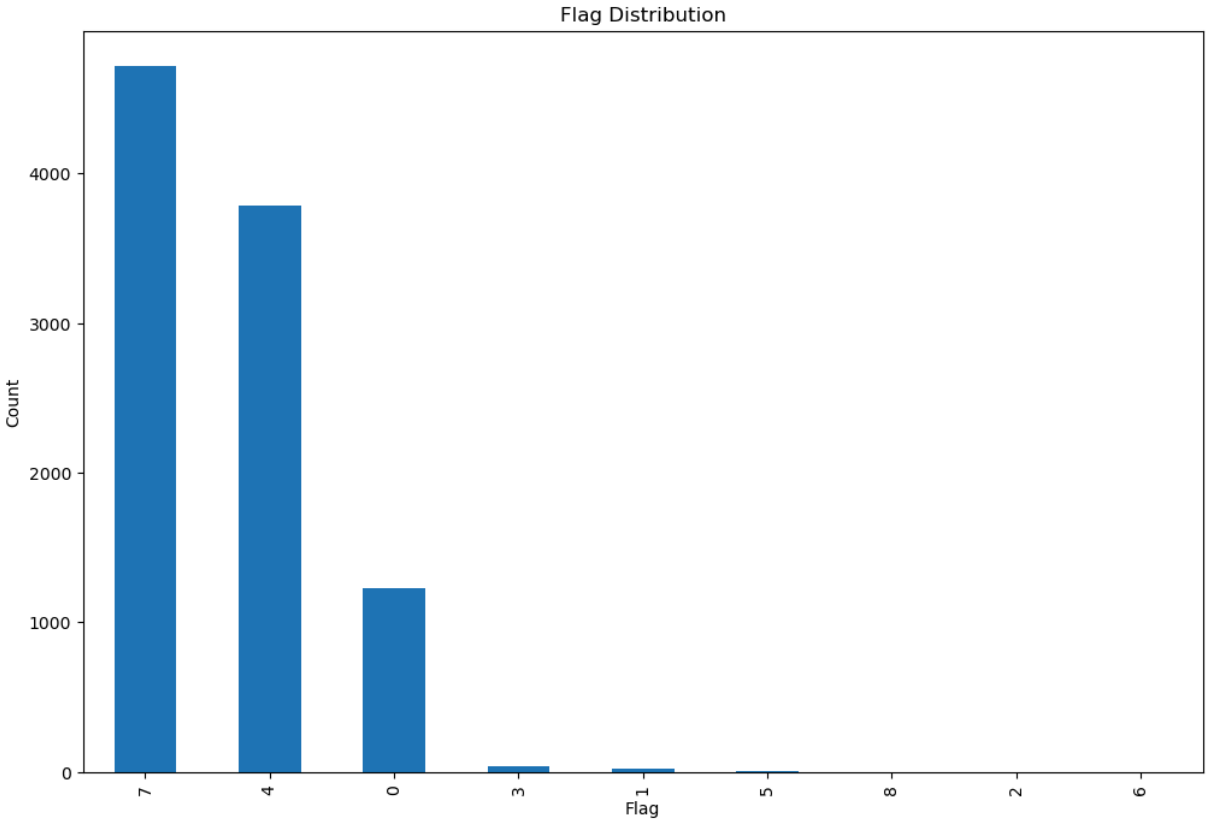
In [37]:
```
plt.scatter(dataset['src_bytes'], dataset['dst_bytes'])
plt.xlabel('Source Bytes')
plt.ylabel('Destination Bytes')
plt.title('Source Bytes vs Destination Bytes')
plt.show()
```



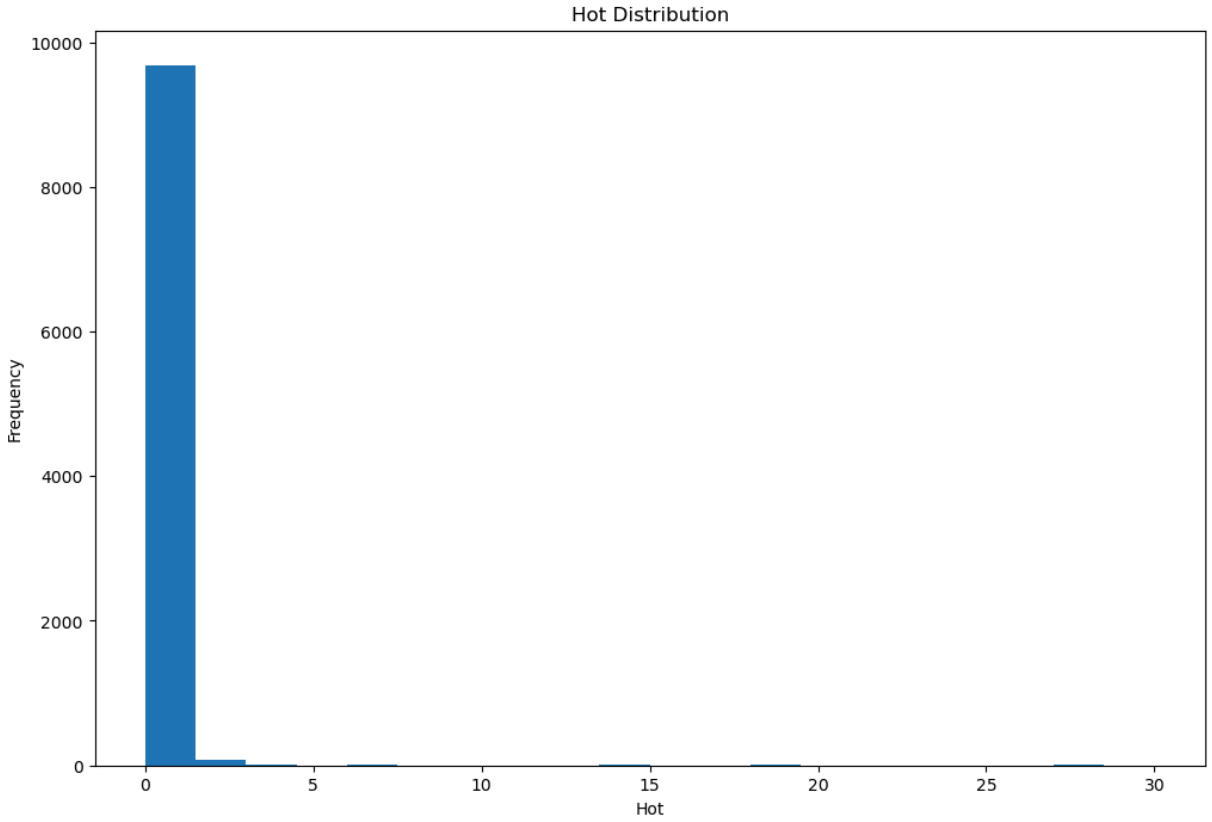
In [38]:
```
dataset.boxplot(column='serror_rate')
plt.ylabel('Serror Rate')
plt.title('Serror Rate Distribution')
plt.show()
```



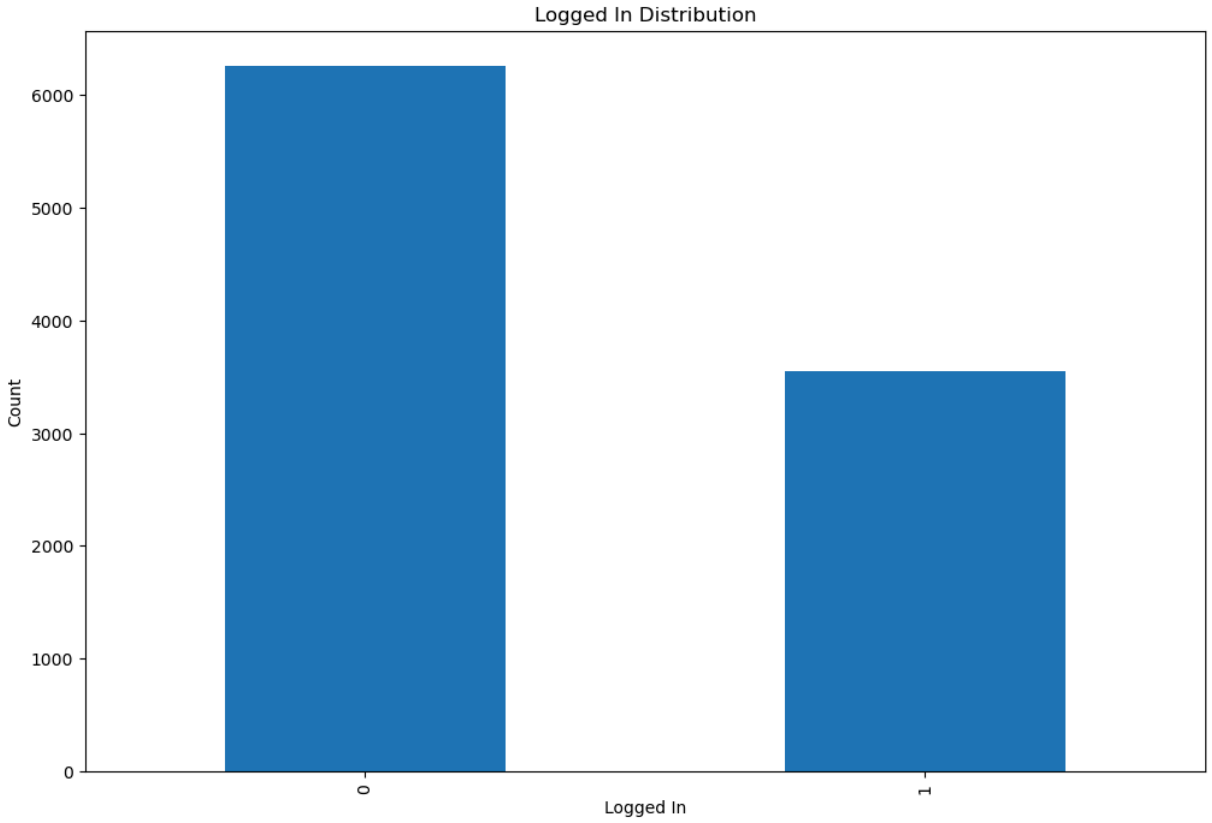
In [39]:
```
dataset['flag'].value_counts().plot(kind='bar')
plt.xlabel('Flag')
plt.ylabel('Count')
plt.title('Flag Distribution')
plt.show()
```



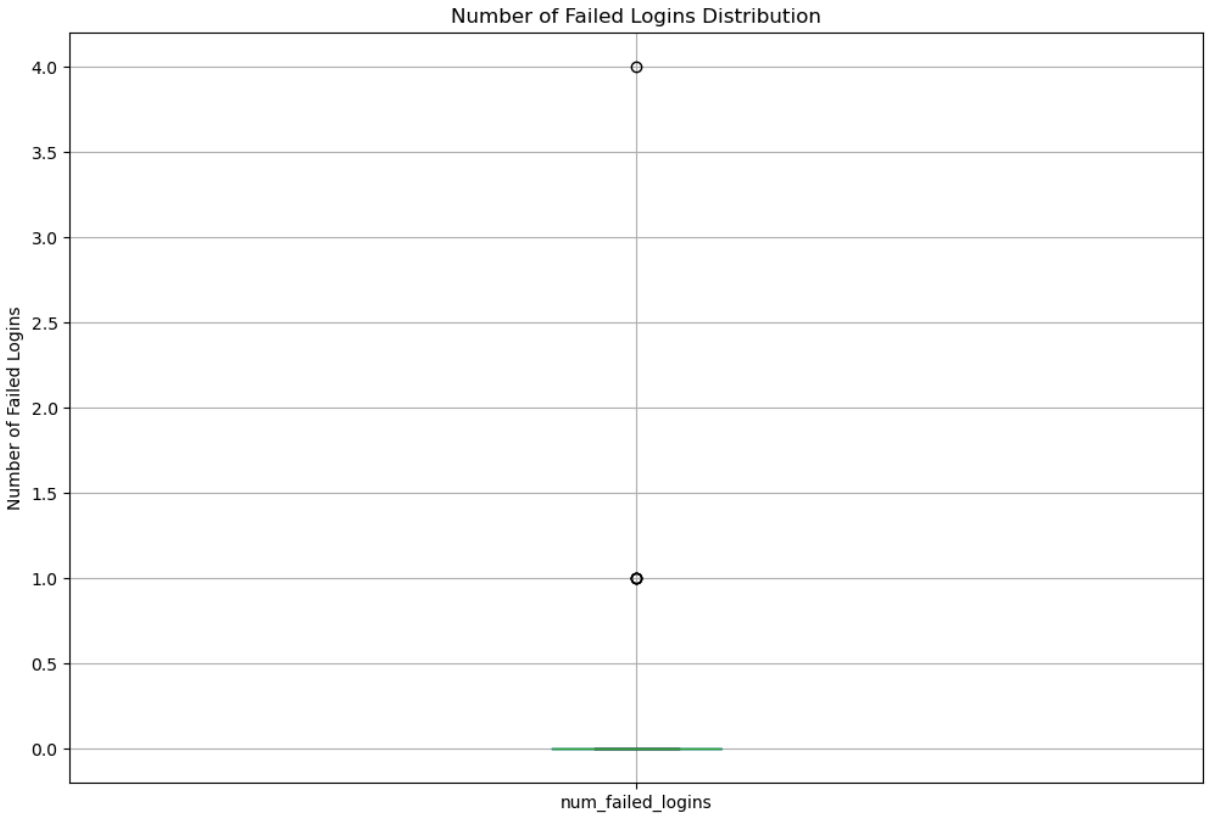
In [40]:
```
dataset['hot'].plot(kind='hist', bins=20)
plt.xlabel('Hot')
plt.ylabel('Frequency')
plt.title('Hot Distribution')
plt.show()
```



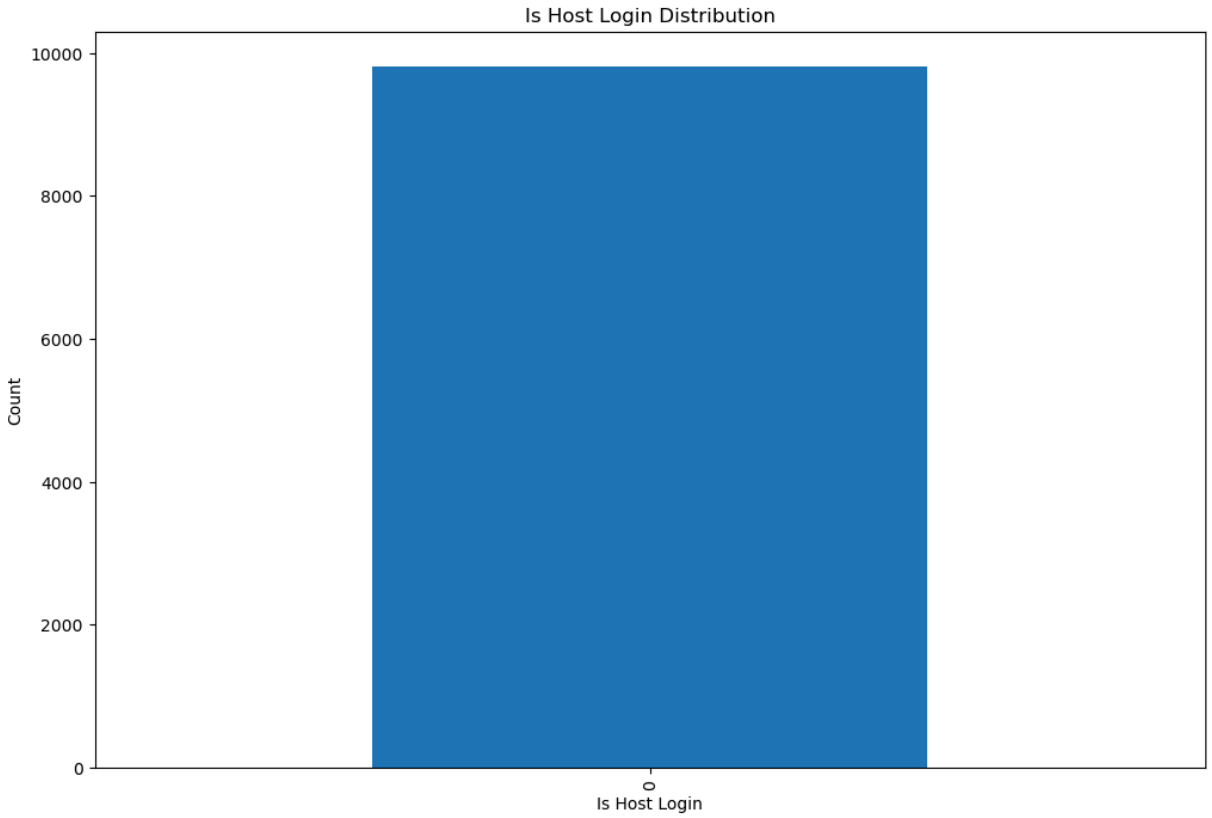
In [41]:
```
dataset['logged_in'].value_counts().plot(kind='bar')
plt.xlabel('Logged In')
plt.ylabel('Count')
plt.title('Logged In Distribution')
plt.show()
```



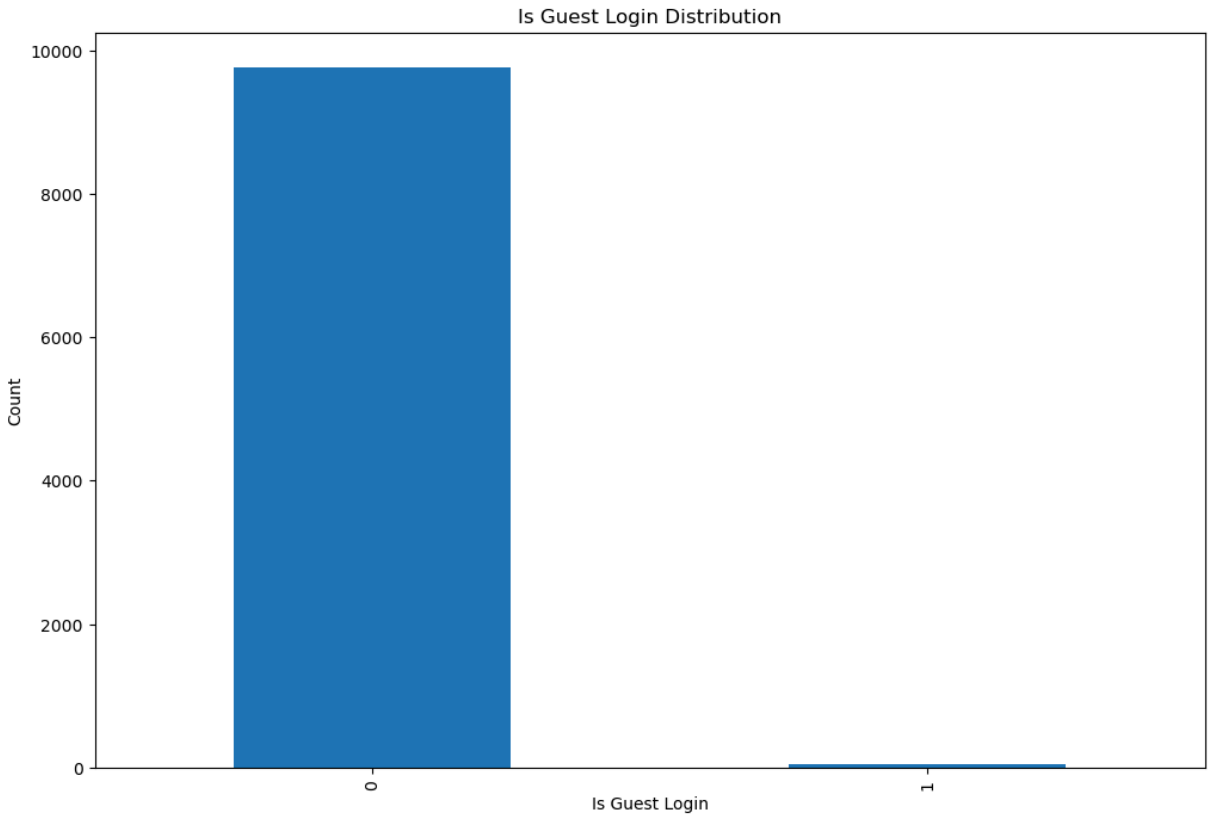
In [42]:
```
dataset.boxplot(column='num_failed_logins')
plt.ylabel('Number of Failed Logins')
plt.title('Number of Failed Logins Distribution')
plt.show()
```



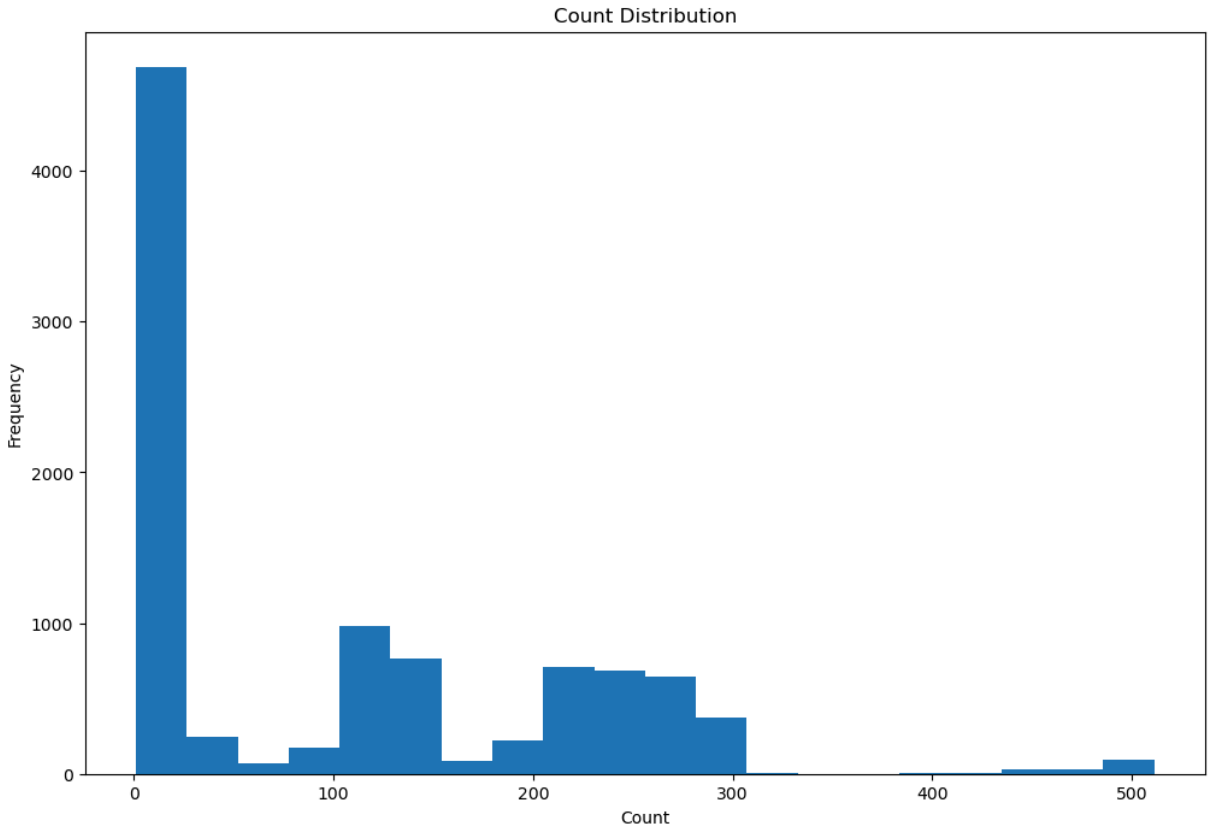
In [43]:
```
dataset['is_host_login'].value_counts().plot(kind='bar')
plt.xlabel('Is Host Login')
plt.ylabel('Count')
plt.title('Is Host Login Distribution')
plt.show()
```



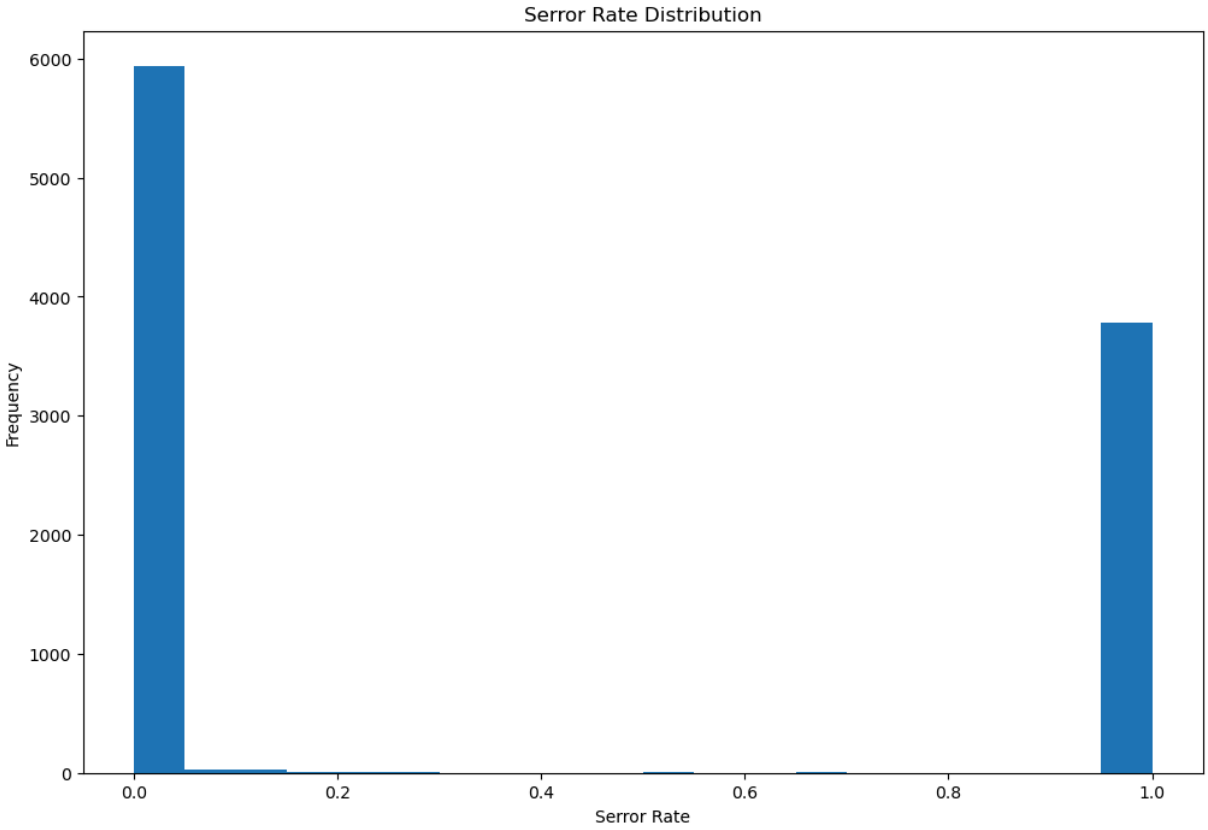
In [44]:
```
dataset['is_guest_login'].value_counts().plot(kind='bar')
plt.xlabel('Is Guest Login')
plt.ylabel('Count')
plt.title('Is Guest Login Distribution')
plt.show()
```



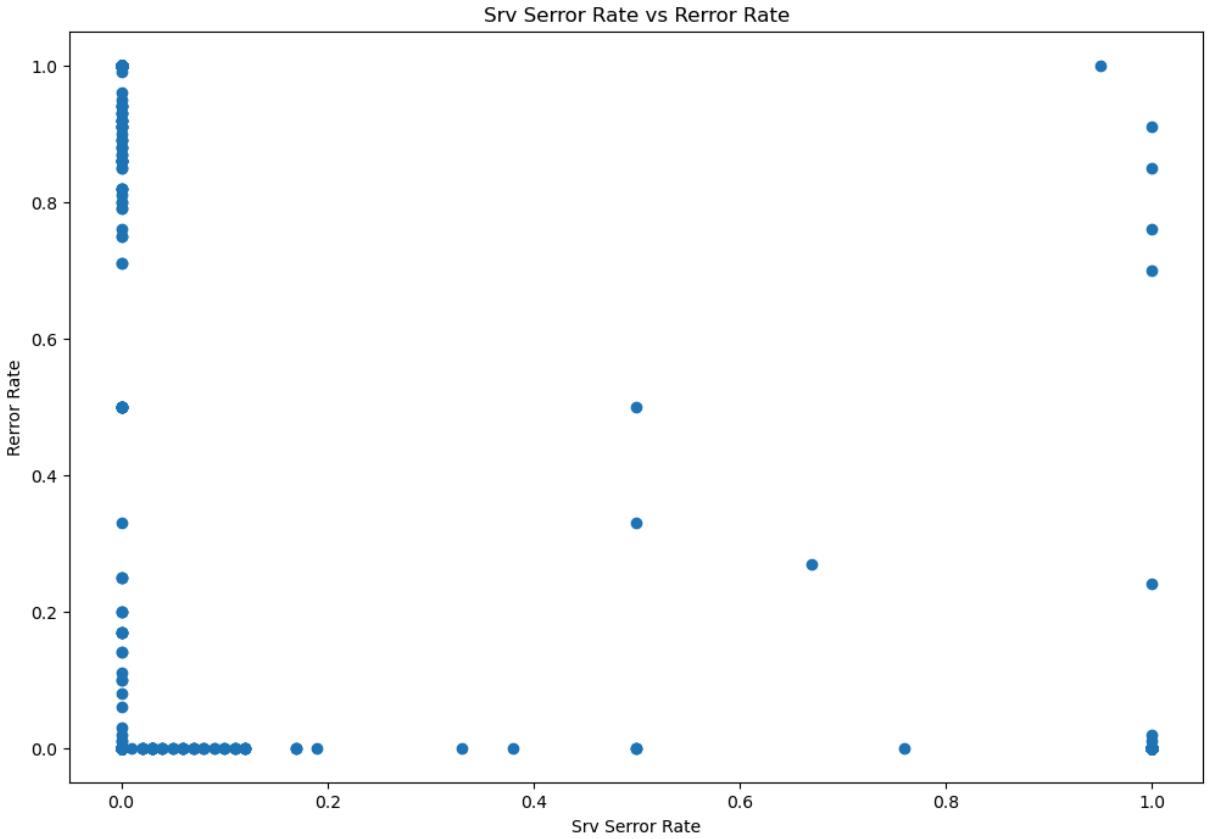
In [45]:
```
dataset['count'].plot(kind='hist', bins=20)
plt.xlabel('Count')
plt.ylabel('Frequency')
plt.title('Count Distribution')
plt.show()
```



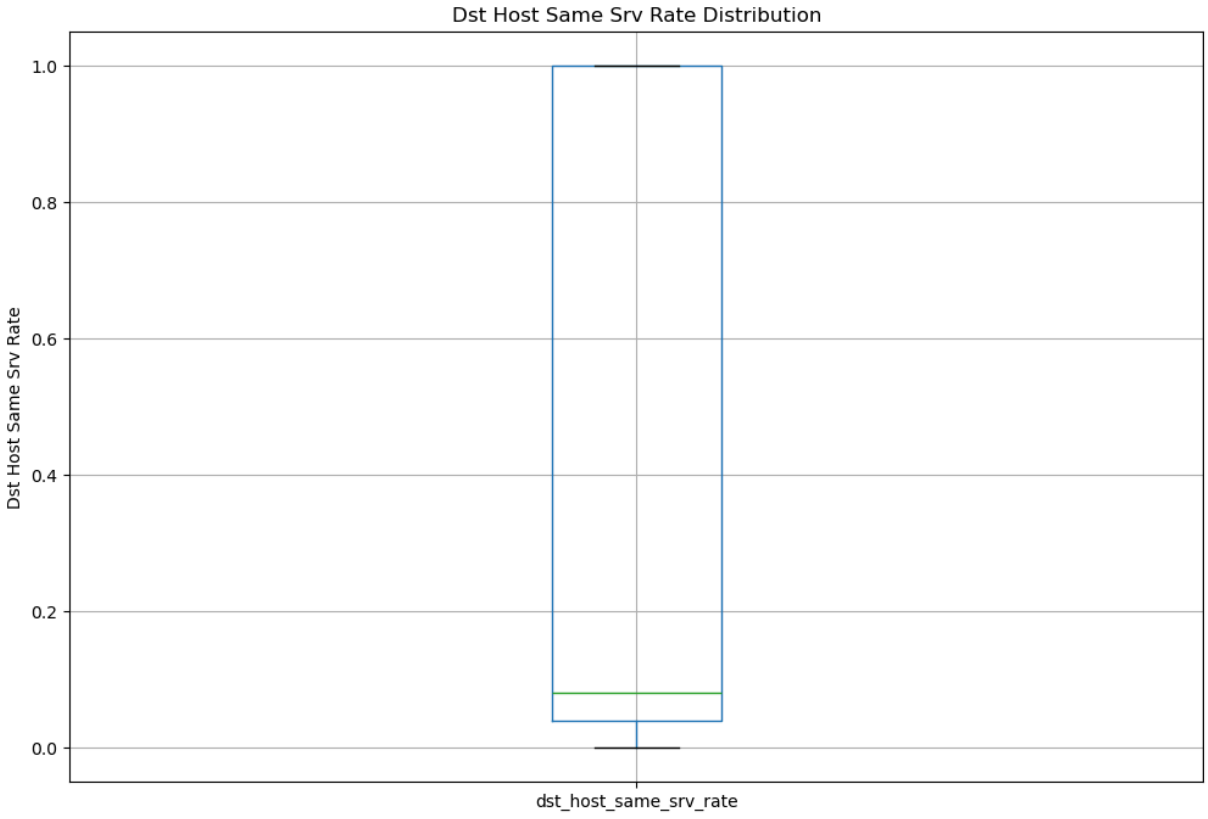
In [46]:
```
dataset['serror_rate'].plot(kind='hist', bins=20)
plt.xlabel('Serror Rate')
plt.ylabel('Frequency')
plt.title('Serror Rate Distribution')
plt.show()
```



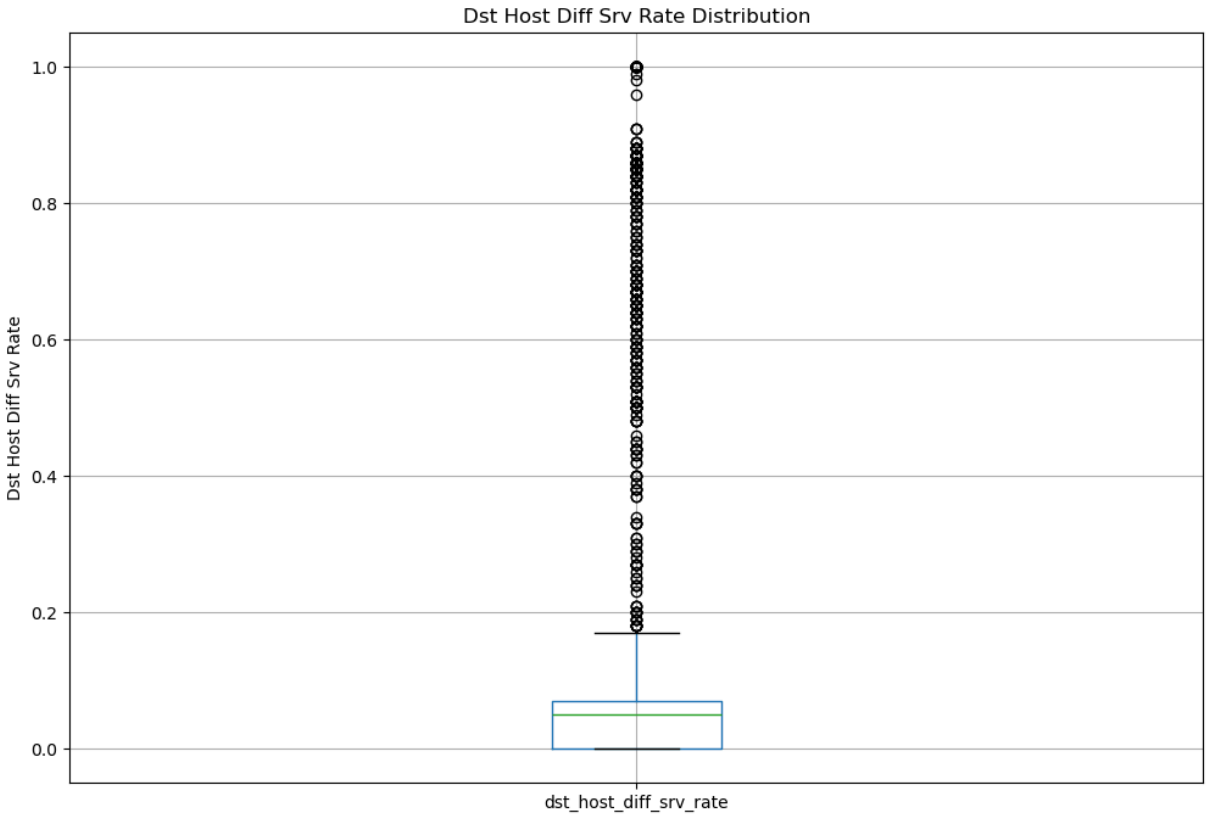
In [47]:
```
plt.scatter(dataset['srv_serror_rate'], dataset['rerror_rate'])
plt.xlabel('Srv Serror Rate')
plt.ylabel('Rerror Rate')
plt.title('Srv Serror Rate vs Rerror Rate')
plt.show()
```



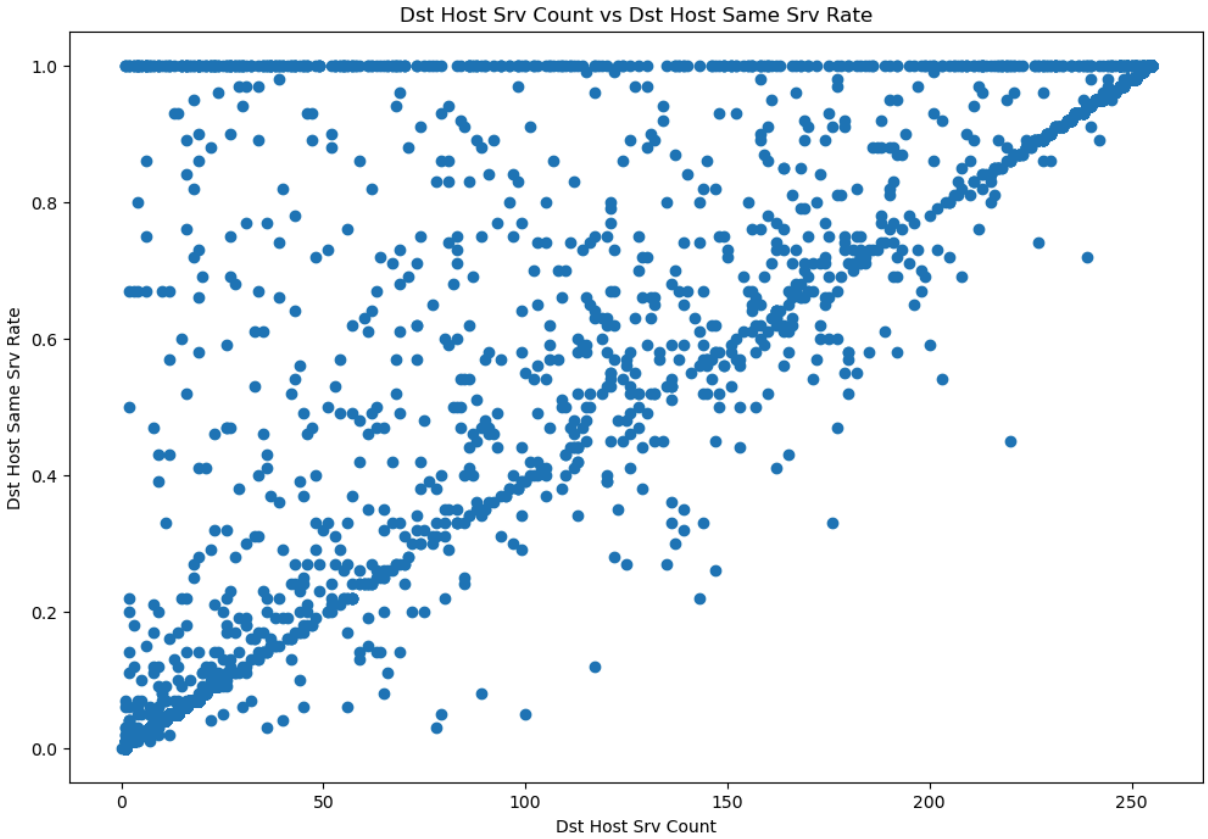
In [48]:
```
dataset.boxplot(column='dst_host_same_srv_rate')
plt.ylabel('Dst Host Same Srv Rate')
plt.title('Dst Host Same Srv Rate Distribution')
plt.show()
```



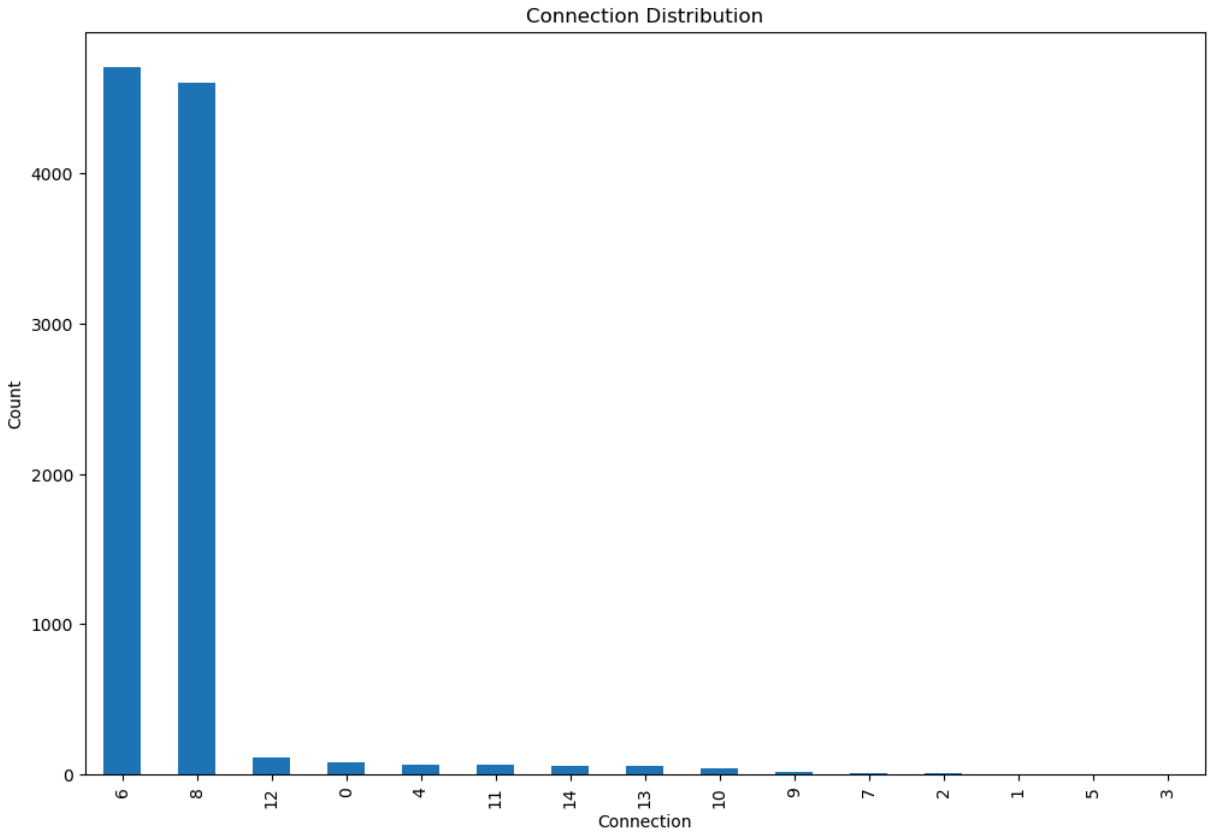
In [49]:
```
dataset.boxplot(column='dst_host_diff_srv_rate')
plt.ylabel('Dst Host Diff Srv Rate')
plt.title('Dst Host Diff Srv Rate Distribution')
plt.show()
```



In [50]:
```
plt.scatter(dataset['dst_host_srv_count'], dataset['dst_host_same_srv_rate'])
plt.xlabel('Dst Host Srv Count')
plt.ylabel('Dst Host Same Srv Rate')
plt.title('Dst Host Srv Count vs Dst Host Same Srv Rate')
plt.show()
```

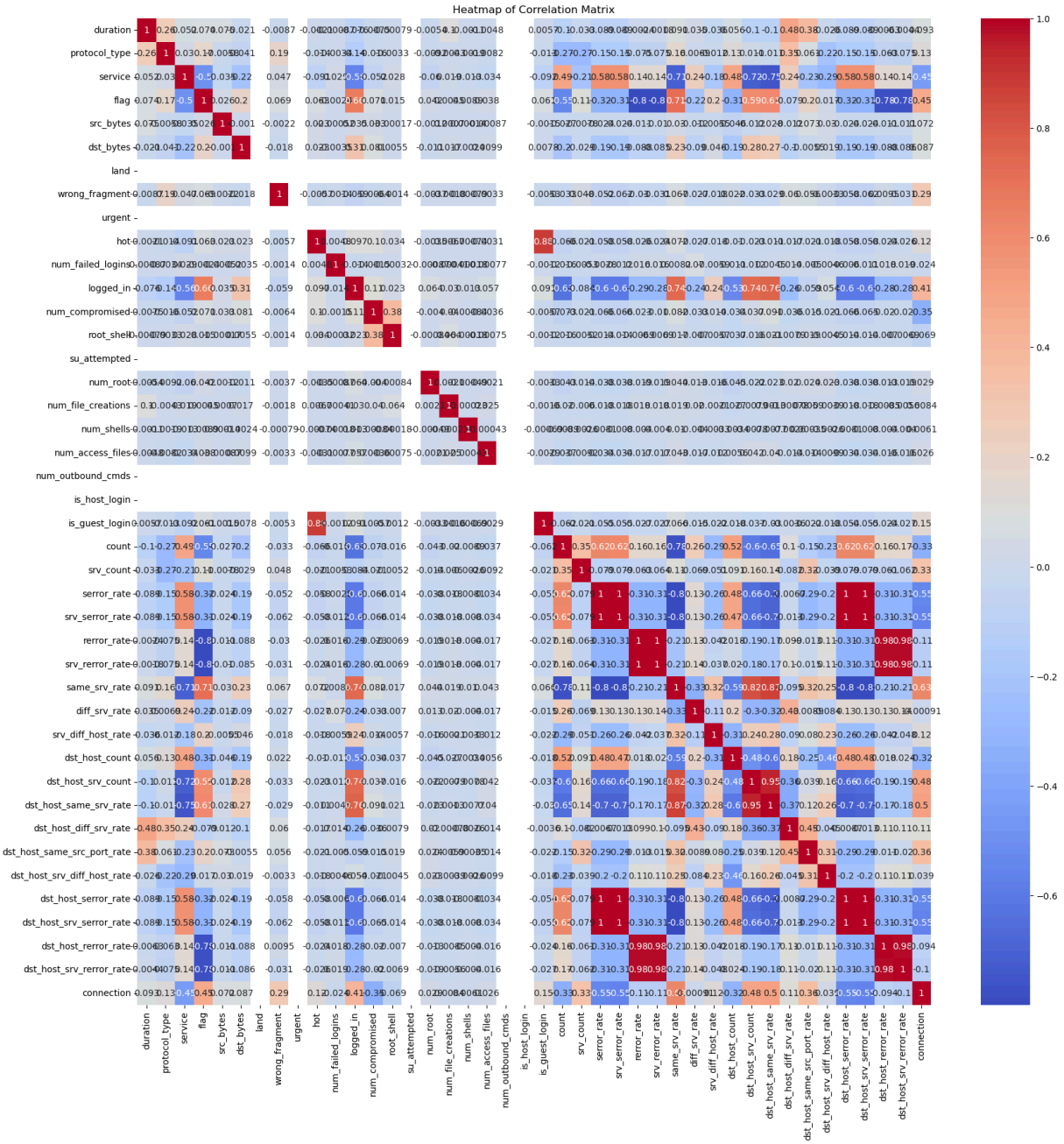


In [51]:
```
dataset['connection'].value_counts().plot(kind='bar')
plt.xlabel('Connection')
plt.ylabel('Count')
plt.title('Connection Distribution')
plt.show()
```



In [52]:
```
corr = dataset.corr()
plt.figure(figsize=(20, 20))
plt.title('Heatmap of Correlation Matrix')
sns.heatmap(corr, annot=True, cmap='coolwarm')
```

Out[52]: <Axes: title={'center': 'Heatmap of Correlation Matrix'}>



Training and Splitting Data set into Test and Train

In [53]:
```
# Splitting the dataset into the Training set and Test set:
X_train, X_test, y_train, y_test = train_test_split(
    transformed_data, target, test_size=0.3, random_state=8)
```

Different Classification Models

Decision Tree Classifier

In [54]:
```
from sklearn.tree import DecisionTreeClassifier

decision_tree = DecisionTreeClassifier()

parameters = [
    {
        "criterion": ["gini", "entropy"],
        "splitter": ["best", "random"],
        "max_features": ["auto", "sqrt", "log2", None],
    }
]


random_search = GridSearchCV(decision_tree, parameters)
random_search.fit(X_train, y_train)
random_search.best_params_
```

Out[54]:
```
{'criterion': 'gini', 'max_features': None, 'splitter': 'random'}
```

In [55]:
```
print("Evaluation for Decision Tree Classifier".center(75, '_'))

decision_tree = DecisionTreeClassifier()
decision_tree.fit(X_train, y_train)

decision_tree_prediction = decision_tree.predict(X_test)
decision_tree_accuracy = accuracy_score(y_test, decision_tree_prediction)
decision_tree_precision = precision_score(
    y_test, decision_tree_prediction, average="weighted")
decision_tree_recall = recall_score(
    y_test, decision_tree_prediction, average="weighted")
decision_tree_f1_score = f1_score(
    y_test, decision_tree_prediction, average="weighted")

print("Prediciton:    ", decision_tree_prediction)
print('_' * 75)

print("Accuracy:" + "\t" + f"{(decision_tree_accuracy * 100)}%")
print("Precision:" + "\t" + f"{(decision_tree_precision * 100)}%")
print("Recall:" + "\t\t" f"{(decision_tree_recall * 100)}%")
print("F1-Score:" + "\t" + f"{(decision_tree_f1_score * 100)}%")
print('_' * 75)
```

```
___________________Evaluation for Decision Tree Classifier___________________
Prediciton:     [6 8 6 ... 6 8 6]
___________________________________________________________________________
Accuracy:       99.28644240570847%
Precision:      99.36386572766747%
Recall:         99.28644240570847%
F1-Score:       99.29126845327966%
___________________________________________________________________________
```

In [56]:
```
print("Confusion Matrix For Decision Tree Classifier - 'Labels Test' & 'Prediction'
print("-" * 93)
print(confusion_matrix(y_test, decision_tree_prediction))
```

```
Confusion Matrix For Decision Tree Classifier - 'Labels Test' & 'Prediction':
--------------------------------------------------------------------------------
---------
[[  28    0    0    0    0    0    0    0    0    0    0    0    0    0]
 [   0    1    0    0    0    0    0    0    0    0    0    0    0    0]
 [   0    0    2    0    0    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0    0    0    1    0    0    0    0    0    0    0]
 [   0    0    0    0   22    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0    0 1400    0    0    0    0    0    0    0    0]
 [   0    0    0    0    1    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0    1    2    0 1387    0    0    2    1    0    4]
 [   0    0    0    0    0    0    0    0    4    0    0    0    0    0]
 [   0    0    0    0    2    0    1    0    0   13    2    0    0    1]
 [   0    0    0    0    0    0    0    0    0    0   15    0    0    0]
 [   0    0    0    0    0    0    0    1    0    0    0   27    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0   10    0]
 [   0    0    0    0    0    0    0    2    0    0    0    0    0   13]]
```

In [57]:
```
print(
    "Classification Report For Decision Tree Classifier - 'Labels Test' & 'Predicti
)
print("-" * 97)
print(classification_report(y_test, decision_tree_prediction))
```

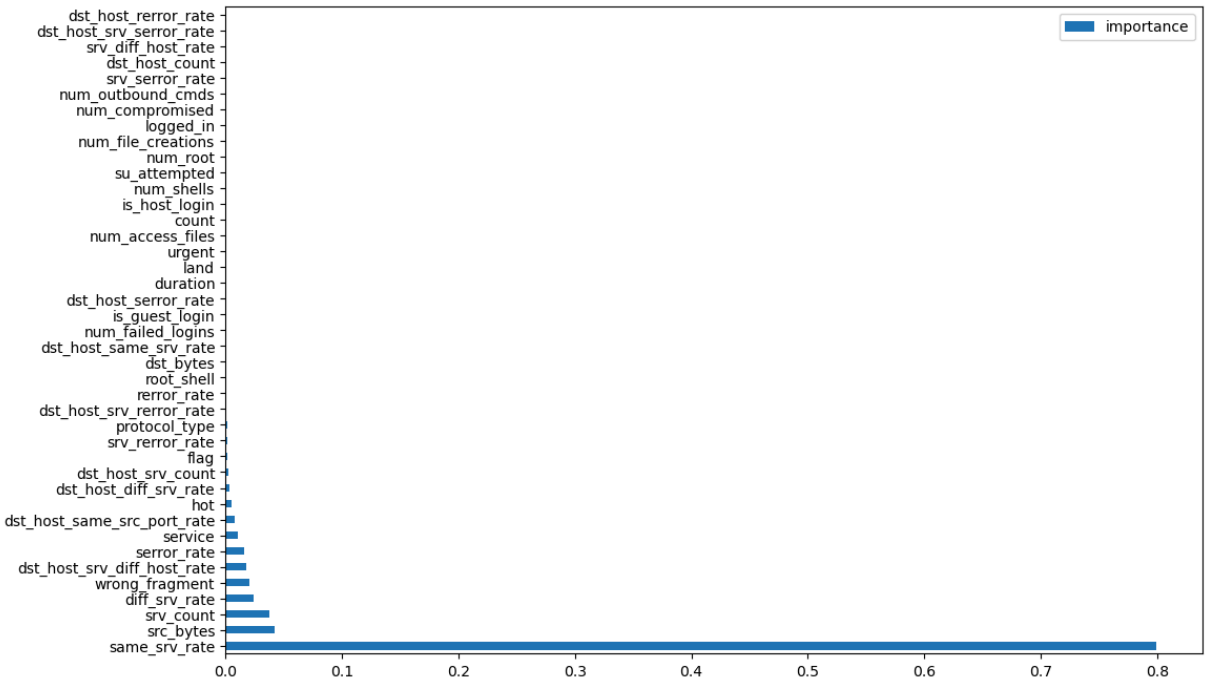
```
Classification Report For Decision Tree Classifier - 'Labels Test' & 'Prediction':
--------------------------------------------------------------------------------
-------------
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        28
           1       1.00      1.00      1.00         1
           2       1.00      1.00      1.00         2
           3       0.00      0.00      0.00         1
           4       0.85      1.00      0.92        22
           6       1.00      1.00      1.00      1400
           7       0.00      0.00      0.00         1
           8       1.00      0.99      1.00      1397
           9       1.00      1.00      1.00         4
          10       1.00      0.68      0.81        19
          11       0.79      1.00      0.88        15
          12       0.96      0.96      0.96        28
          13       1.00      1.00      1.00        10
          14       0.72      0.87      0.79        15

    accuracy                           0.99      2943
   macro avg       0.81      0.82      0.81      2943
weighted avg       0.99      0.99      0.99      2943
```

In [58]:
```
feature_importance = pd.DataFrame(decision_tree.feature_importances_, index=transfo
                                  'importance']).sort_values('importance', ascendin
print(feature_importance)
```

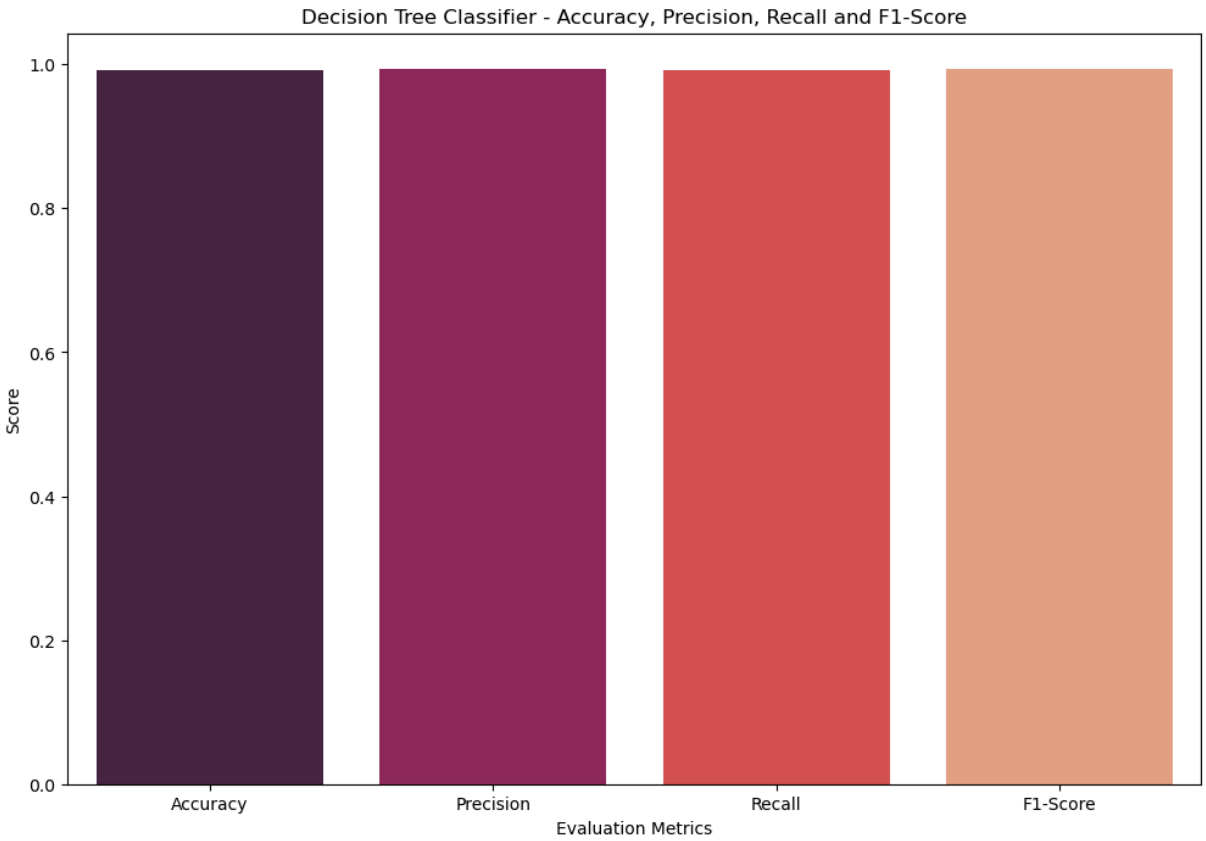
	importance
same_srv_rate	0.798909
src_bytes	0.042739
srv_count	0.037928
diff_srv_rate	0.024470
wrong_fragment	0.021005
dst_host_srv_diff_host_rate	0.018326
serror_rate	0.015743
service	0.010329
dst_host_same_src_port_rate	0.007835
hot	0.005156
dst_host_diff_srv_rate	0.003682
dst_host_srv_count	0.002446
flag	0.001901
srv_rerror_rate	0.001512
protocol_type	0.001445
dst_host_srv_rerror_rate	0.001218
rerror_rate	0.001150
root_shell	0.001056
dst_bytes	0.000877
dst_host_same_srv_rate	0.000873
num_failed_logins	0.000703
is_guest_login	0.000521
dst_host_serror_rate	0.000177
duration	0.000000
land	0.000000
urgent	0.000000
num_access_files	0.000000
count	0.000000
is_host_login	0.000000
num_shells	0.000000
su_attempted	0.000000
num_root	0.000000
num_file_creations	0.000000
logged_in	0.000000
num_compromised	0.000000
num_outbound_cmds	0.000000
srv_serror_rate	0.000000
dst_host_count	0.000000
srv_diff_host_rate	0.000000
dst_host_srv_serror_rate	0.000000
dst_host_rerror_rate	0.000000

In [59]:
```
feature_importance.plot(kind="barh")
```

Out[59]: <Axes: >



In [60]:
```
season = ["Accuracy", "Precision", "Recall", "F1-Score"]
season_score = [decision_tree_accuracy, decision_tree_precision,
                decision_tree_recall, decision_tree_f1_score]

sns.barplot(x=season, y=season_score, palette="rocket")
plt.title("Decision Tree Classifier - Accuracy, Precision, Recall and F1-Score")
plt.xlabel("Evaluation Metrics")
plt.ylabel("Score")
plt.show()
```



Random Forest Classifier

In [61]:
```
from sklearn.ensemble import RandomForestClassifier

random_forest = RandomForestClassifier()

parameters = [
    {
        "n_estimators": [10, 50, 100, 150, 200],
        "criterion": ["gini", "entropy"],
        "max_features": ["auto", "sqrt", "log2", None],
    }
]

random_search = GridSearchCV(random_forest, parameters)
random_search.fit(X_train, y_train)
random_search.best_params_
```

Out[61]:
```
{'criterion': 'gini', 'max_features': 'sqrt', 'n_estimators': 150}
```

In [62]:
```
print("Evaluation for Random Forest Classifier".center(75, '_'))

random_forest_classifier = RandomForestClassifier(
    n_estimators=100, random_state=0)
random_forest_classifier.fit(X_train, y_train)

random_forest_classifier_prediction = random_forest_classifier.predict(X_test)
random_forest_classifier_accuracy = accuracy_score(
```

```
    y_test, random_forest_classifier_prediction)
random_forest_classifier_precision = precision_score(
    y_test, random_forest_classifier_prediction, average="weighted")
random_forest_classifier_recall = recall_score(
    y_test, random_forest_classifier_prediction, average="weighted")
random_forest_classifier_f1_score = f1_score(
    y_test, random_forest_classifier_prediction, average="weighted")

print("Prediciton:    ", random_forest_classifier_prediction)
print('_' * 75)

print("Accuracy:" + "\t" + f"{(random_forest_classifier_accuracy * 100)}%")
print("Precision:" + "\t" + f"{(random_forest_classifier_precision * 100)}%")
print("Recall:" + "\t\t" f"{(random_forest_classifier_recall * 100)}%")
print("F1-Score:" + "\t" + f"{(random_forest_classifier_f1_score * 100)}%")
print('_' * 75)
```

```
___________________Evaluation for Random Forest Classifier___________________
Prediciton:     [6 8 6 ... 6 8 6]
___________________________________________________________________________
Accuracy:       99.79612640163099%
Precision:      99.7339901912306%
Recall:         99.79612640163099%
F1-Score:       99.75609448621405%
___________________________________________________________________________
```

In [63]:
```
print("Confusion Matrix For Random Forest Classifier - 'Labels Test' & 'Prediction'
print("-" * 93)
print(confusion_matrix(y_test, random_forest_classifier_prediction))
```

```
Confusion Matrix For Random Forest Classifier - 'Labels Test' & 'Prediction':
-----------------------------------------------------------------------------
---------
[[  28    0    0    0    0    0    0    0    0    0    0    0    0    0]
 [   0    1    0    0    0    0    0    0    0    0    0    0    0    0]
 [   0    0    1    0    0    0    0    1    0    0    0    0    0    0]
 [   0    0    0    0    0    1    0    0    0    0    0    0    0    0]
 [   0    0    0    0   22    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0    0 1400    0    0    0    0    0    0    0    0]
 [   0    0    0    0    1    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0 1396    0    0    0    0    0    1]
 [   0    0    0    0    0    0    0    0    4    0    0    0    0    0]
 [   0    0    0    0    0    1    0    0    0   17    1    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0   15    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0   28    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0   10    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0   15]]
```

In [64]:
```
print(
    "Classification Report For Random Forest Classifier - 'Labels Test' & 'Predicti
)
print("-" * 97)
print(classification_report(y_test, random_forest_classifier_prediction))
```

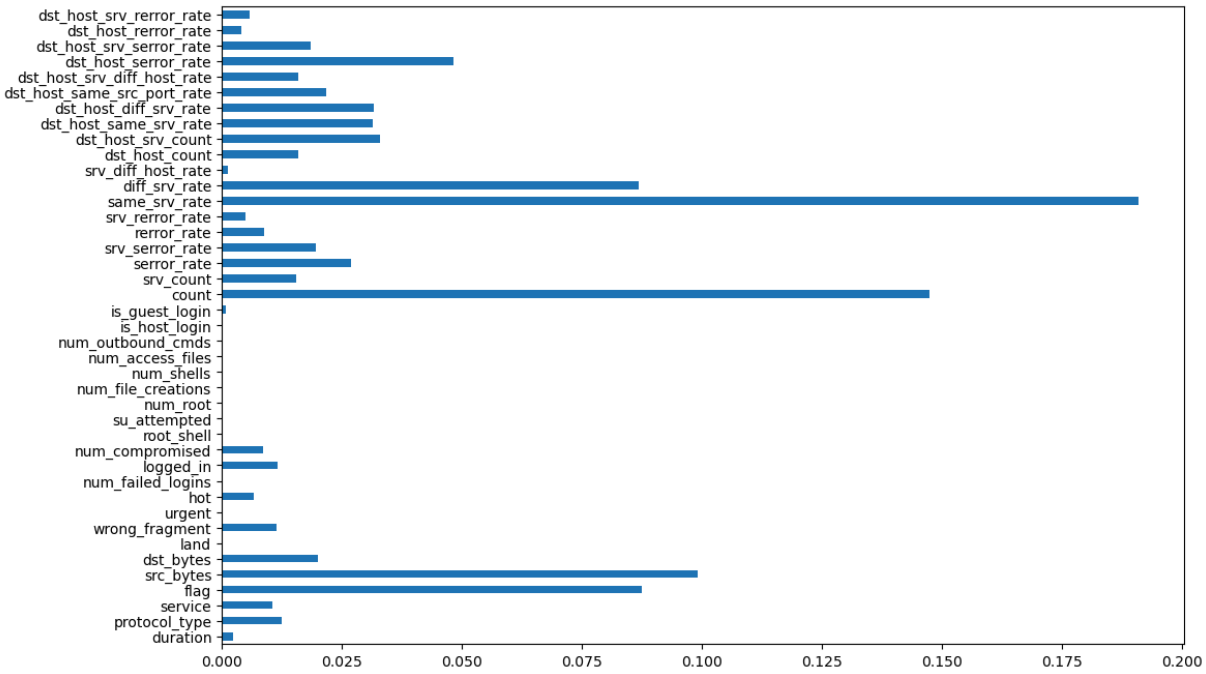
```
Classification Report For Random Forest Classifier - 'Labels Test' & 'Prediction':
--------------------------------------------------------------------------------
-------------
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        28
           1       1.00      1.00      1.00         1
           2       1.00      0.50      0.67         2
           3       0.00      0.00      0.00         1
           4       0.96      1.00      0.98        22
           6       1.00      1.00      1.00      1400
           7       0.00      0.00      0.00         1
           8       1.00      1.00      1.00      1397
           9       1.00      1.00      1.00         4
          10       1.00      0.89      0.94        19
          11       0.94      1.00      0.97        15
          12       1.00      1.00      1.00        28
          13       1.00      1.00      1.00        10
          14       0.94      1.00      0.97        15

    accuracy                           1.00      2943
   macro avg       0.84      0.81      0.82      2943
weighted avg       1.00      1.00      1.00      2943
```

In [65]:
```
feature_importance = pd.Series(
    random_forest_classifier.feature_importances_, index=transformed_data.columns)
feature_importance
```

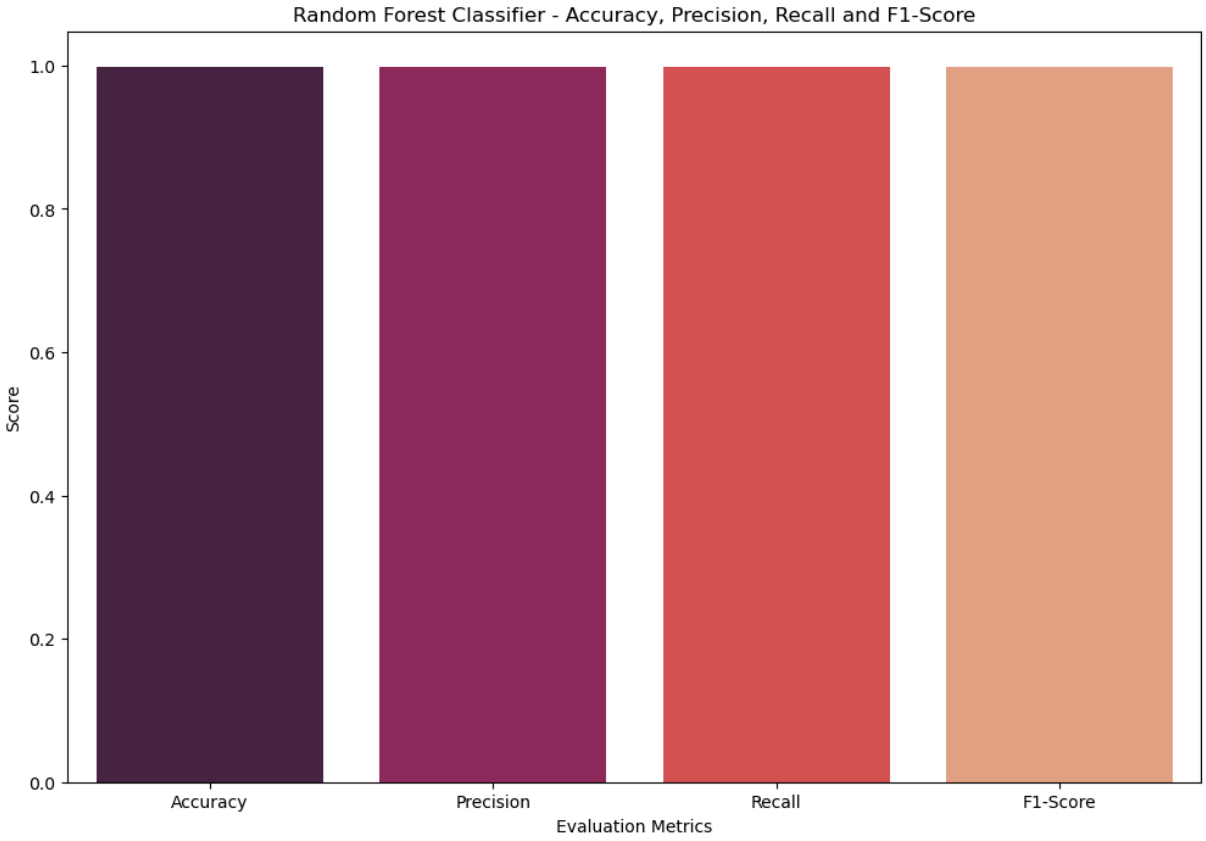
Out[65]:
```
duration                        2.387013e-03
protocol_type                   1.255539e-02
service                         1.043770e-02
flag                            8.746110e-02
src_bytes                       9.899253e-02
dst_bytes                       1.996021e-02
land                            0.000000e+00
wrong_fragment                  1.143382e-02
urgent                          0.000000e+00
hot                             6.585637e-03
num_failed_logins               1.072640e-04
logged_in                       1.168597e-02
num_compromised                 8.612565e-03
root_shell                      1.560736e-04
su_attempted                    0.000000e+00
num_root                        4.979625e-05
num_file_creations              2.682452e-05
num_shells                      1.781059e-09
num_access_files                4.978477e-10
num_outbound_cmds               0.000000e+00
is_host_login                   0.000000e+00
is_guest_login                  9.206832e-04
count                           1.473580e-01
srv_count                       1.540508e-02
serror_rate                     2.694794e-02
srv_serror_rate                 1.954778e-02
rerror_rate                     8.880578e-03
srv_rerror_rate                 4.903293e-03
same_srv_rate                   1.908373e-01
diff_srv_rate                   8.688497e-02
srv_diff_host_rate              1.280935e-03
dst_host_count                  1.595450e-02
dst_host_srv_count              3.298203e-02
dst_host_same_srv_rate          3.152031e-02
dst_host_diff_srv_rate          3.172841e-02
dst_host_same_src_port_rate     2.170974e-02
dst_host_srv_diff_host_rate     1.595320e-02
dst_host_serror_rate            4.822595e-02
dst_host_srv_serror_rate        1.860171e-02
dst_host_rerror_rate            4.082593e-03
dst_host_srv_rerror_rate        5.823023e-03
dtype: float64
```

In [66]:
```
feature_importance.plot(kind="barh")
```

Out[66]:
```
<Axes: >
```



In [67]:
```
season = ["Accuracy", "Precision", "Recall", "F1-Score"]
season_score = [random_forest_classifier_accuracy, random_forest_classifier_precisi
                random_forest_classifier_recall, random_forest_classifier_f1_score]

sns.barplot(x=season, y=season_score, palette="rocket")
plt.title("Random Forest Classifier - Accuracy, Precision, Recall and F1-Score")
plt.xlabel("Evaluation Metrics")
plt.ylabel("Score")
plt.show()
```



K-Nearest Neighbors

In [68]:
```
# apply grid search to knn
from sklearn.model_selection import GridSearchCV
from sklearn.neighbors import KNeighborsClassifier

knn_model = KNeighborsClassifier()

knn_param_grid = [
    {
        'n_neighbors': [3, 5, 7, 9, 11, 13, 15],
        'weights': ['uniform', 'distance'],
        'metric': ['euclidean', 'manhattan']
    }
]

knn_grid_search = GridSearchCV(knn_model, knn_param_grid)
knn_grid_search.fit(X_train, y_train)
knn_grid_search.best_params_
```

Out[68]:
```
{'metric': 'euclidean', 'n_neighbors': 3, 'weights': 'distance'}
```

In [69]:
```
print("Evaluation for K-Nearest Neighbors".center(75, '_'))

k_nearest_neighbors = KNeighborsClassifier(n_neighbors=5)
k_nearest_neighbors.fit(X_train, y_train)

k_nearest_neighbors_prediction = k_nearest_neighbors.predict(X_test)
k_nearest_neighbors_accuracy = accuracy_score(
    y_test, k_nearest_neighbors_prediction)
k_nearest_neighbors_precision = precision_score(
    y_test, k_nearest_neighbors_prediction, average="weighted")
k_nearest_neighbors_recall = recall_score(
    y_test, k_nearest_neighbors_prediction, average="weighted")
k_nearest_neighbors_f1_score = f1_score(
    y_test, k_nearest_neighbors_prediction, average="weighted")

print("Prediciton:    ", k_nearest_neighbors_prediction)
print('_' * 75)

print("Accuracy:" + "\t" + f"{(k_nearest_neighbors_accuracy * 100)}%")
print("Precision:" + "\t" + f"{(k_nearest_neighbors_precision * 100)}%")
print("Recall:" + "\t\t" f"{(k_nearest_neighbors_recall * 100)}%")
print("F1-Score:" + "\t" + f"{(k_nearest_neighbors_f1_score * 100)}%")
print('_' * 75)
```

```
_____________________Evaluation for K-Nearest Neighbors_____________________
Prediciton:     [6 8 6 ... 6 8 6]
___________________________________________________________________________
Accuracy:       99.11654774040095%
Precision:      98.99119621093692%
Recall:         99.11654774040095%
F1-Score:       98.9979778342897%
___________________________________________________________________________
```

In [70]:
```
print("Confusion Matrix For K-Nearest Neighbors - 'Labels Test' & 'Prediction':")
print('-' * 93)
print(confusion_matrix(y_test, k_nearest_neighbors_prediction))
```

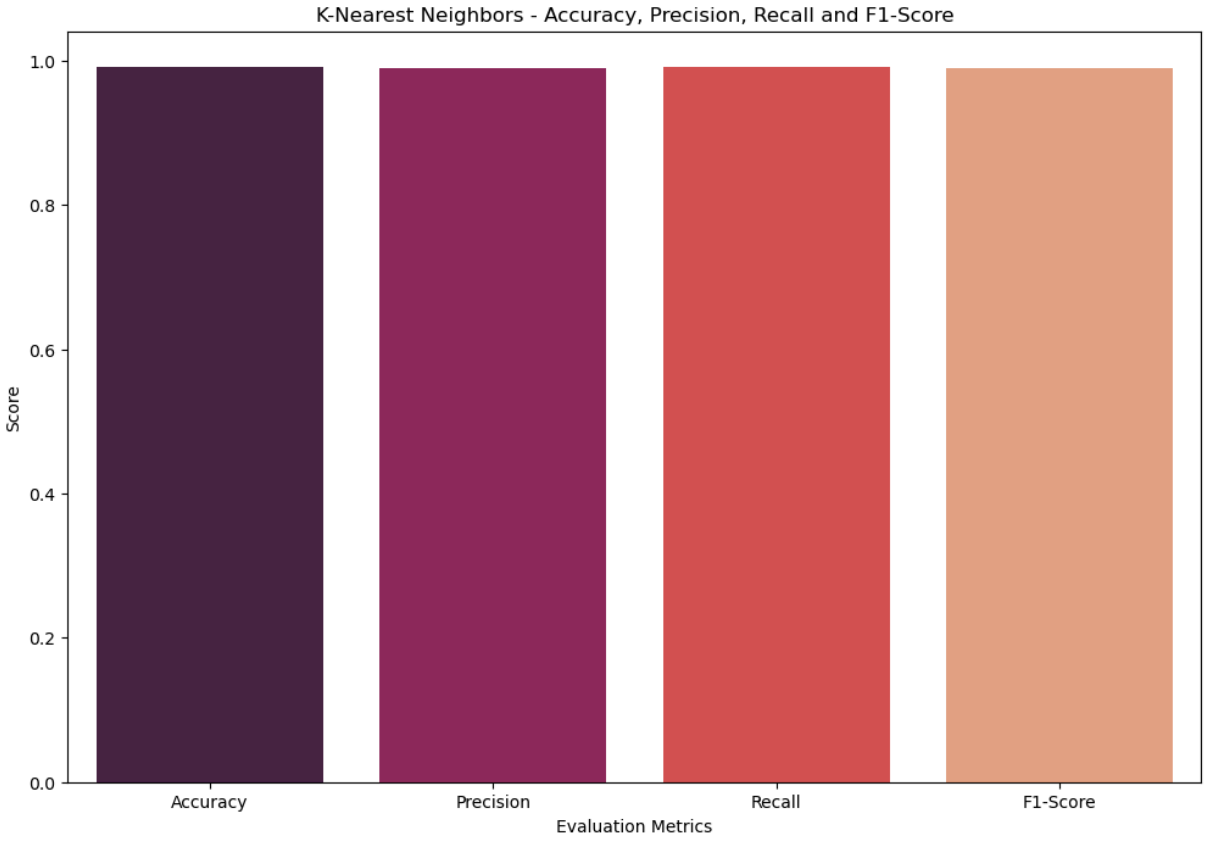
```
Confusion Matrix For K-Nearest Neighbors - 'Labels Test' & 'Prediction':
---------------------------------------------------------------------------------------------
---------
[[  27    0    0    0    0    0    0    1    0    0    0    0    0    0]
 [   1    0    0    0    0    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    2    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    1    0    0    0    0    0    0]
 [   0    0    0    0   22    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0    0 1400    0    0    0    0    0    0    0    0]
 [   0    0    0    0    1    0    0    0    0    0    0    0    0    0]
 [   2    0    0    0    0    0    0 1390    0    0    0    1    0    4]
 [   0    0    0    0    0    0    0    2    2    0    0    0    0    0]
 [   0    0    0    0    0    3    0    2    0   11    3    0    0    0]
 [   0    0    0    0    0    0    0    2    0    0   13    0    0    0]
 [   0    0    0    0    0    0    0    1    0    0    0   27    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0   10    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0   15]]
```

In [71]:
```
print("Classification Report For K-Nearest Neighbors - 'Labels Test' & 'Prediction'
print('-' * 97)
print(classification_report(y_test, k_nearest_neighbors_prediction))
```

```
Classification Report For K-Nearest Neighbors - 'Labels Test' & 'Prediction':
---------------------------------------------------------------------------------------------
-------------
              precision    recall  f1-score   support

           0       0.90      0.96      0.93        28
           1       0.00      0.00      0.00         1
           2       0.00      0.00      0.00         2
           3       0.00      0.00      0.00         1
           4       0.96      1.00      0.98        22
           6       1.00      1.00      1.00      1400
           7       0.00      0.00      0.00         1
           8       0.99      0.99      0.99      1397
           9       1.00      0.50      0.67         4
          10       1.00      0.58      0.73        19
          11       0.81      0.87      0.84        15
          12       0.96      0.96      0.96        28
          13       1.00      1.00      1.00        10
          14       0.79      1.00      0.88        15

    accuracy                           0.99      2943
   macro avg       0.67      0.63      0.64      2943
weighted avg       0.99      0.99      0.99      2943
```

In [72]:
```
season = ["Accuracy", "Precision", "Recall", "F1-Score"]
season_score = [k_nearest_neighbors_accuracy, k_nearest_neighbors_precision,
                k_nearest_neighbors_recall, k_nearest_neighbors_f1_score]

sns.barplot(x=season, y=season_score, palette="rocket")
```

```
plt.title("K-Nearest Neighbors - Accuracy, Precision, Recall and F1-Score")
plt.xlabel("Evaluation Metrics")
plt.ylabel("Score")
plt.show()
```



Ada Boost

In [73]:
```
# apply grid search to ada boost

from sklearn.ensemble import AdaBoostClassifier

ada_boost_model = AdaBoostClassifier()

ada_boost_param_grid = [
    {
        'n_estimators': [50, 100, 150, 200, 250, 300, 350, 400, 450, 500],
        'learning_rate': [0.001, 0.01, 0.1, 1.0, 10.0],
        'algorithm': ['SAMME', 'SAMME.R']
    }
]

ada_boost_grid_search = GridSearchCV(ada_boost_model, ada_boost_param_grid)
ada_boost_grid_search.fit(X_train, y_train)
ada_boost_grid_search.best_params_
```

Out[73]:
```
{'algorithm': 'SAMME', 'learning_rate': 1.0, 'n_estimators': 100}
```

In [74]:
```
print("Evaluation for Ada Boost".center(75, "_"))

ada_boast = AdaBoostClassifier(n_estimators=100, random_state=0)
ada_boast.fit(X_train, y_train)

ada_boost_prediction = ada_boast.predict(X_test)
ada_boost_accuracy = accuracy_score(y_test, ada_boost_prediction)
ada_boost_precision = precision_score(y_test, ada_boost_prediction, average="weight
ada_boost_recall = recall_score(y_test, ada_boost_prediction, average="weighted")
ada_boost_f1_score = f1_score(y_test, ada_boost_prediction, average="weighted")

print("Prediciton:    ", ada_boost_prediction)
print("_" * 75)

print("Accuracy:" + "\t" + f"{(ada_boost_accuracy * 100)}%")
print("Precision:" + "\t" + f"{(ada_boost_precision * 100)}%")
print("Recall:" + "\t\t" f"{(ada_boost_recall * 100)}%")
print("F1-Score:" + "\t" + f"{(ada_boost_f1_score * 100)}%")
print("_" * 75)
```

```
_____________________________Evaluation for Ada Boost_____________________________
Prediciton:     [6 8 6 ... 6 8 6]
___________________________________________________________________________
Accuracy:       94.22358137954468%
Precision:      89.7850953107695%
Recall:         94.22358137954468%
F1-Score:       91.91177978962602%
___________________________________________________________________________
```

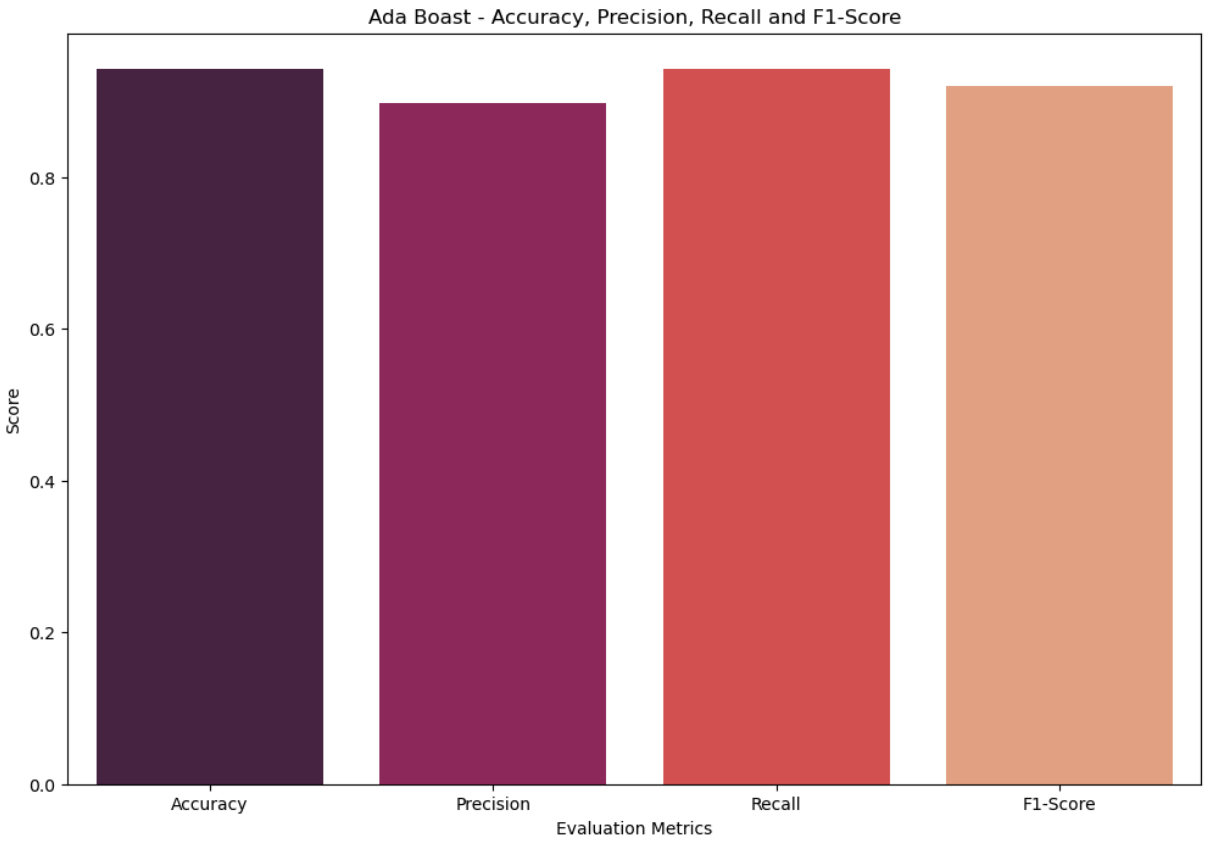
In [75]:
```
print("Confusion Matrix For Ada Boast - 'Labels Test' & 'Prediction':")
print('-' * 93)
print(confusion_matrix(y_test, ada_boost_prediction))
```

```
Confusion Matrix For Ada Boast - 'Labels Test' & 'Prediction':
-----------------------------------------------------------------------------
---------
[[   0    0    0    0    0    0    0   28    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0    0    0    1]
 [   0    0    0    0    0    0    0    0    0    0    2    0    0    0]
 [   0    0    0    0    0    0    0    1    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0   22    0    0    0    0    0    0]
 [   0    0    0    0    0 1385    0   15    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    1    0    0    0    0    0    0]
 [   0    0    0    0    0    9    0 1388    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    4    0    0    0    0    0    0]
 [   0    0    0    0    0    3    0   16    0    0    0    0    0    0]
 [   0    0    0    0    0   14    0    1    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0   28    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0   10    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0   15    0    0    0    0    0    0]]
```

In [76]:
```
print("Classification Report For Ada Boast - 'Labels Test' & 'Prediction':")
print('-' * 97)
print(classification_report(y_test, ada_boost_prediction))
```

```
Classification Report For Ada Boast - 'Labels Test' & 'Prediction':
--------------------------------------------------------------------------------
-------------
              precision    recall  f1-score   support

           0       0.00      0.00      0.00        28
           1       0.00      0.00      0.00         1
           2       0.00      0.00      0.00         2
           3       0.00      0.00      0.00         1
           4       0.00      0.00      0.00        22
           6       0.98      0.99      0.99      1400
           7       0.00      0.00      0.00         1
           8       0.91      0.99      0.95      1397
           9       0.00      0.00      0.00         4
          10       0.00      0.00      0.00        19
          11       0.00      0.00      0.00        15
          12       0.00      0.00      0.00        28
          13       0.00      0.00      0.00        10
          14       0.00      0.00      0.00        15

    accuracy                           0.94      2943
   macro avg       0.13      0.14      0.14      2943
weighted avg       0.90      0.94      0.92      2943
```

In [77]:
```
season = ["Accuracy", "Precision", "Recall", "F1-Score"]
season_score = [ada_boost_accuracy, ada_boost_precision,
                ada_boost_recall, ada_boost_f1_score]

sns.barplot(x=season, y=season_score, palette="rocket")
plt.title("Ada Boast - Accuracy, Precision, Recall and F1-Score")
plt.xlabel("Evaluation Metrics")
plt.ylabel("Score")
plt.show()
```



Gradient Boosting

In [102…

```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import GridSearchCV

gradient_boost_model = GradientBoostingClassifier(random_state=42)

gradient_boost_param_grid = {
    'learning_rate': [0.05, 0.1],
    'n_estimators': [100, 200],
    'max_depth': [3, 5],
    'criterion': ['friedman_mse']
}

gradient_boost_grid_search = GridSearchCV(
    estimator=gradient_boost_model,
    param_grid=gradient_boost_param_grid,
    cv=3,
    scoring='accuracy',
    n_jobs=-1
)

gradient_boost_grid_search.fit(X_train, y_train)

print(gradient_boost_grid_search.best_params_)
print(gradient_boost_grid_search.best_score_)
```

```
{'criterion': 'friedman_mse', 'learning_rate': 0.05, 'max_depth': 3, 'n_estimators':
200}
0.9960660513513596
```

In [103…

```
print("Evaluation for Gradient Boosting".center(75, "_"))

gradient_boosting = GradientBoostingClassifier(
    n_estimators=100, learning_rate=1.0, max_depth=1, random_state=0
)
gradient_boosting.fit(X_train, y_train)

gradient_boosting_prediction = gradient_boosting.predict(X_test)
gradient_boosting_accuracy = accuracy_score(y_test, gradient_boosting_prediction)
gradient_boosting_precision = precision_score(
    y_test, gradient_boosting_prediction, average="weighted"
)
gradient_boosting_recall = recall_score(
    y_test, gradient_boosting_prediction, average="weighted"
)
gradient_boosting_f1_score = f1_score(
    y_test, gradient_boosting_prediction, average="weighted"
)

print("Prediciton:    ", gradient_boosting_prediction)
print("_" * 75)

print("Accuracy:" + "\t" + f"{(gradient_boosting_accuracy * 100)}%")
print("Precision:" + "\t" + f"{(gradient_boosting_precision * 100)}%")
print("Recall:" + "\t\t" f"{(gradient_boosting_recall * 100)}%")
print("F1-Score:" + "\t" + f"{(gradient_boosting_f1_score * 100)}%")
print("_" * 75)
```

```
_________________________Evaluation for Gradient Boosting_________________________
Prediciton:     [10 10 10 ... 10 10 10]
___________________________________________________________________________
Accuracy:       1.4276002719238612%
Precision:      1.070717309145664%
Recall:         1.4276002719238612%
F1-Score:       1.1690131561714865%
___________________________________________________________________________
```

In [104…

```
print("Confusion Matrix For Gradient Boosting - 'Labels Test' & 'Prediction':")
print('-' * 93)
print(confusion_matrix(y_test, gradient_boosting_prediction))
```

```
Confusion Matrix For Gradient Boosting - 'Labels Test' & 'Prediction':
--------------------------------------------------------------------------------
---------
[[   0    0    0    0    0    0    0    0    0    0   25    0    0]
 [   0    0    0    0    0    0    0    0    0    0    1    0    0]
 [   0    0    0    0    0    0    0    0    0    0    1    0    0]
 [   0    0    0   19    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0 1412    0    0]
 [   0    0    0    0    0    0    0    0    0    0    1    0    0]
 [   0    0    0    2    0    0    0    0    0    0 1380    0    0]
 [   0    0    0    1    0    0    0    0    0    0    0    3    0]
 [   0    0    0    0    0    0    0    0    3    0    8    0    0]
 [   0    0    0    0    0    0    0    0   18    0    1    0    0]
 [   0    0    0    0    0    0    0    0    0    0    4    0   29]
 [   0    0    0    0    0    0    0    0    0    0    0   16    0]
 [   0    0    0    0    0    0    0    0    0    0   18    0    0]]
```

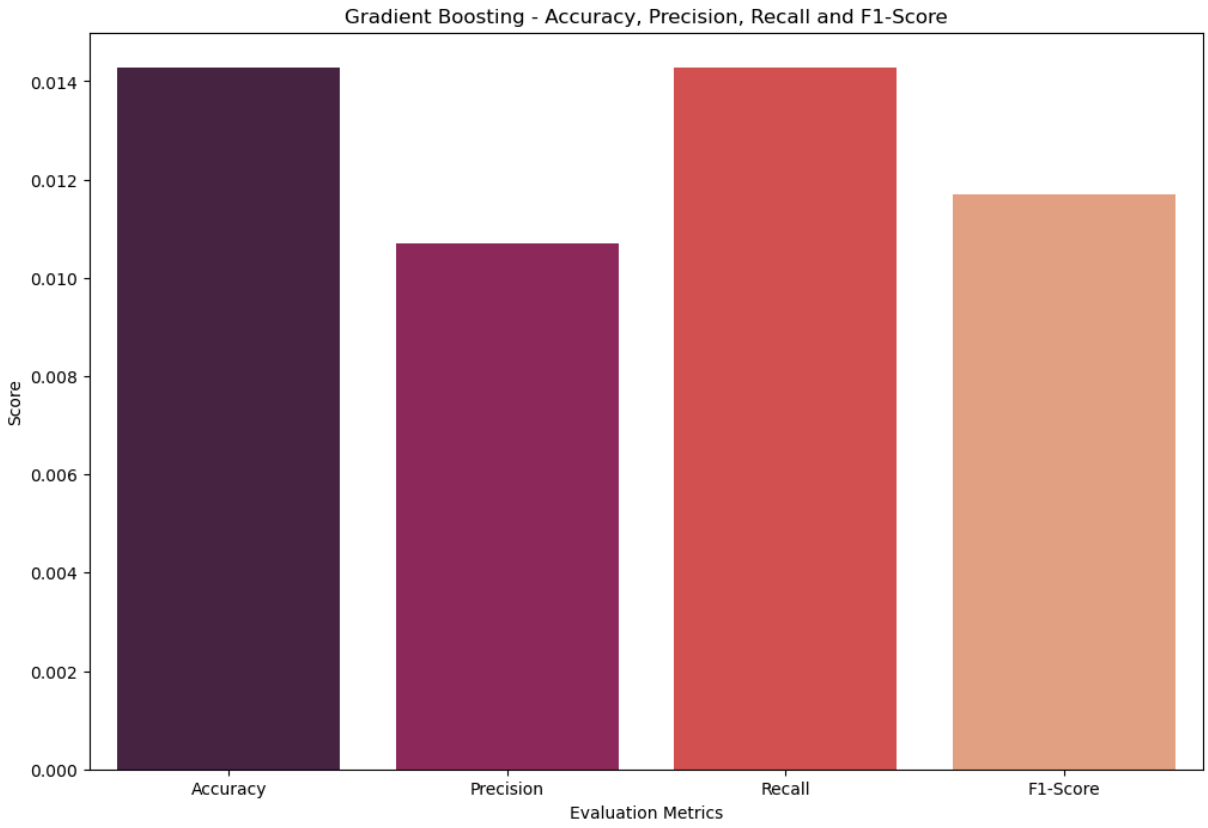
In [105…

```
print("Classification Report For Gradient Boosting - 'Labels Test' & 'Prediction':"
print('-' * 97)
print(classification_report(y_test, gradient_boosting_prediction))
```

```
Classification Report For Gradient Boosting - 'Labels Test' & 'Prediction':
--------------------------------------------------------------------------------
-------------
              precision    recall  f1-score   support

           0       0.00      0.00      0.00        25
           1       0.00      0.00      0.00         1
           2       0.00      0.00      0.00         1
           3       0.86      1.00      0.93        19
           4       0.00      0.00      0.00      1412
           5       0.00      0.00      0.00         1
           6       0.00      0.00      0.00      1382
           7       0.00      0.00      0.00         4
           8       0.14      0.27      0.19        11
           9       0.00      0.00      0.00        19
          10       0.00      0.12      0.00        33
          11       0.84      1.00      0.91        16
          12       0.00      0.00      0.00        18

    accuracy                           0.01      2942
   macro avg       0.14      0.18      0.16      2942
weighted avg       0.01      0.01      0.01      2942
```

In [106…

```
season = ["Accuracy", "Precision", "Recall", "F1-Score"]
season_score = [gradient_boosting_accuracy, gradient_boosting_precision,
                gradient_boosting_recall, gradient_boosting_f1_score]

sns.barplot(x=season, y=season_score, palette="rocket")
plt.title("Gradient Boosting - Accuracy, Precision, Recall and F1-Score")
plt.xlabel("Evaluation Metrics")
plt.ylabel("Score")
plt.show()
```



Hist Gradient Boosting

In [84]:
```
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.model_selection import GridSearchCV

hist_model = HistGradientBoostingClassifier(random_state=42)

param_grid = {
    "learning_rate": [0.01, 0.1],
    "max_iter": [100, 200],
    "max_depth": [3, 5]
}

grid = GridSearchCV(
    hist_model,
    param_grid,
    cv=3,
    n_jobs=-1,
    scoring="accuracy"
)

grid.fit(X_train, y_train)

print(grid.best_params_)
```

```
{'learning_rate': 0.01, 'max_depth': 5, 'max_iter': 200}
```

In [85]:
```
print("Evaluation for Hist Gradient Boosting".center(75, "_"))
```

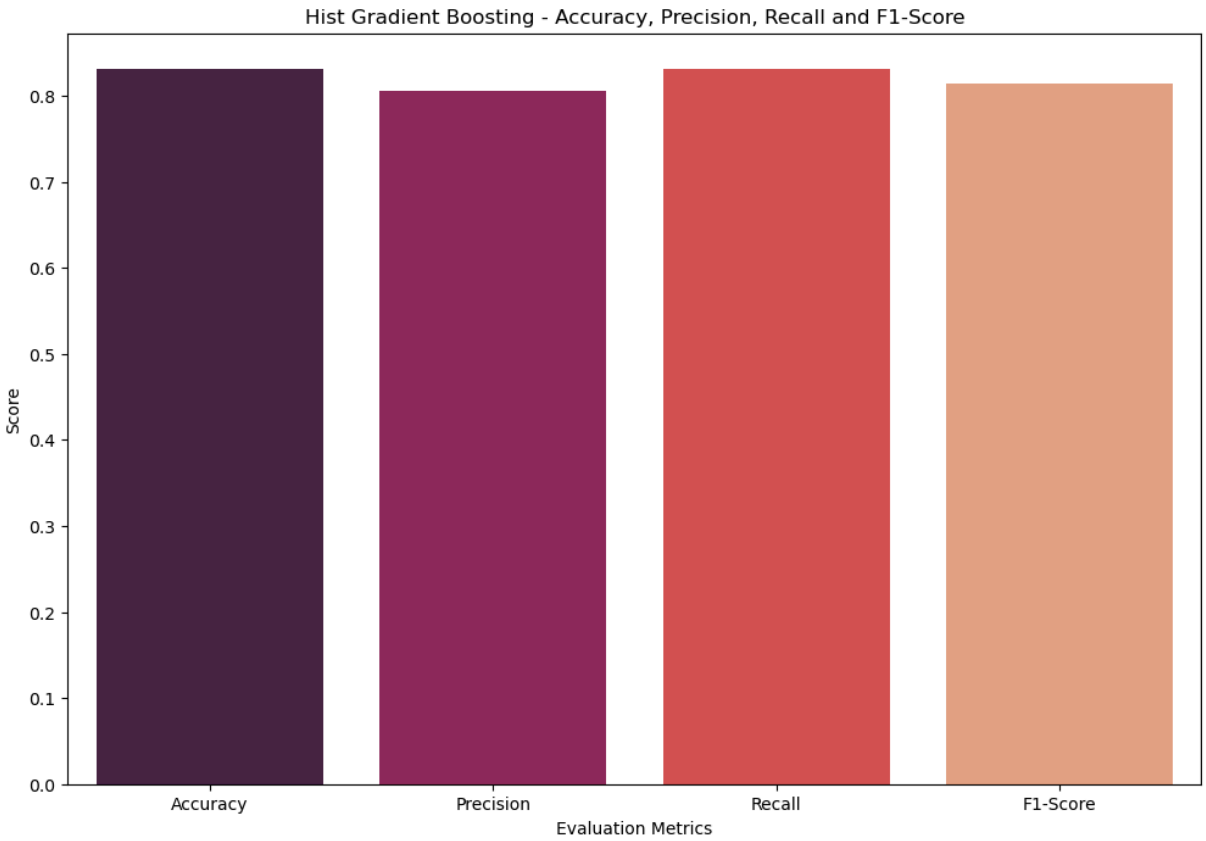
```
hist_gradient_boosting = HistGradientBoostingClassifier(
    loss="log_loss", learning_rate=0.1, max_iter=100, max_leaf_nodes=31, random_sta
)
hist_gradient_boosting.fit(X_train, y_train)

hist_gradient_boosting_prediction = hist_gradient_boosting.predict(X_test)
hist_gradient_boosting_accuracy = accuracy_score(
    y_test, hist_gradient_boosting_prediction
)
hist_gradient_boosting_precision = precision_score(
    y_test, hist_gradient_boosting_prediction, average="weighted"
)
hist_gradient_boosting_recall = recall_score(
    y_test, hist_gradient_boosting_prediction, average="weighted"
)
hist_gradient_boosting_f1_score = f1_score(
    y_test, hist_gradient_boosting_prediction, average="weighted"
)

print("Prediciton:    ", hist_gradient_boosting_prediction)
print("_" * 75)

print("Accuracy:" + "\t" + f"{(hist_gradient_boosting_accuracy * 100)}%")
print("Precision:" + "\t" + f"{(hist_gradient_boosting_precision * 100)}%")
print("Recall:" + "\t\t" f"{(hist_gradient_boosting_recall * 100)}%")
print("F1-Score:" + "\t" + f"{(hist_gradient_boosting_f1_score * 100)}%")
print("_" * 75)
```

```
____________________Evaluation for Hist Gradient Boosting____________________
Prediciton:     [6 8 6 ... 6 6 6]
___________________________________________________________________________
Accuracy:       83.11247026843357%
Precision:      80.6064688921018%
Recall:         83.11247026843357%
F1-Score:       81.4489140901371%
___________________________________________________________________________
```

In [86]:
```
print("Confusion Matrix For Hist Gradient Boosting - 'Labels Test' & 'Prediction':"
print('-' * 93)
print(confusion_matrix(y_test, hist_gradient_boosting_prediction))
```

```
Confusion Matrix For Hist Gradient Boosting - 'Labels Test' & 'Prediction':
--------------------------------------------------------------------------------
---------
[[   0    0    0    0    0    0    0   28    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    1    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    2    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    1    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0   22    0    0    0    0    0    0]
 [   0    0    0    0   10 1335    0   35    0    1    4    0   14    1]
 [   0    0    0    0    0    0    0    1    0    0    0    0    0    0]
 [   0    0    0    0    0  298    0 1099    0    0    0    0    0    0]
 [   0    0    0    0    0    1    0    3    0    0    0    0    0    0]
 [   0    0    0    0    0    3    0   16    0    0    0    0    0    0]
 [   0    0    0    0    0    4    0    0    0    0   11    0    0    0]
 [   0    0    0    0    0    3    0   25    0    0    0    0    0    0]
 [   0    0    0    0    0    8    0    1    0    0    0    0    1    0]
 [   0    0    0    0    0    0    0   15    0    0    0    0    0    0]]
```

In [87]:
```
season = ["Accuracy", "Precision", "Recall", "F1-Score"]
season_score = [
    hist_gradient_boosting_accuracy,
    hist_gradient_boosting_precision,
    hist_gradient_boosting_recall,
    hist_gradient_boosting_f1_score,
]

sns.barplot(x=season, y=season_score, palette="rocket")
plt.title("Hist Gradient Boosting - Accuracy, Precision, Recall and F1-Score")
plt.xlabel("Evaluation Metrics")
plt.ylabel("Score")
plt.show()
```



XGBoost Classifier

In [88]:
```
import pandas as pd

# Count samples in each class
counts = dataset["connection"].value_counts()

# Keep only classes having at least 2 samples
valid_classes = counts[counts >= 2].index

dataset = dataset[dataset["connection"].isin(valid_classes)].reset_index(drop=True)

print(dataset.shape)
print(dataset["connection"].value_counts())
```

```
(9805, 42)
connection
6     4706
8     4605
12     111
0       82
4       63
11      63
14      59
13      53
10      38
9       13
7        5
2        4
1        3
Name: count, dtype: int64
```

In [89]:
```
X = dataset.drop("connection", axis=1)
y = dataset["connection"]
```

In [90]:
```
from sklearn.preprocessing import LabelEncoder

obj_cols = X.select_dtypes(include="object").columns

for col in obj_cols:
    le = LabelEncoder()
    X[col] = le.fit_transform(X[col])
```

In [91]:
```
target_encoder = LabelEncoder()
y = target_encoder.fit_transform(y)

print("Classes:", target_encoder.classes_)
print("Encoded labels:", sorted(set(y)))
```

```
Classes: [ 0  1  2  4  6  7  8  9 10 11 12 13 14]
Encoded labels: [np.int64(0), np.int64(1), np.int64(2), np.int64(3), np.int64(4), n
p.int64(5), np.int64(6), np.int64(7), np.int64(8), np.int64(9), np.int64(10), np.int
64(11), np.int64(12)]
```

In [92]:
```
print(X.shape)
print(len(y))
```

```
(9805, 41)
9805
```

In [93]:
```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print(X_scaled.shape)
```

```
(9805, 41)
```

In [94]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
```

```
    X_scaled,
    y,
    test_size=0.3,
    random_state=8,
    stratify=y
)

print(np.unique(y_train))
print(np.unique(y_test))
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12]
[ 0  1  2  3  4  5  6  7  8  9 10 11 12]
```

In [95]:
```
from xgboost import XGBClassifier

model = XGBClassifier(
    objective="multi:softprob",
    eval_metric="mlogloss",
    random_state=42
)

model.fit(X_train, y_train)
```

Out[95]:

XGBClassifier

```
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds
=None,
              enable_categorical=True, eval_metric='mlogloss',
              feature_types=None, feature_weights=None, gamma=None,
              grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=None, max_bin
=None,
```

In [96]:
```
print("Evaluation for XGBoost".center(75, '_'))

xgboost = XGBClassifier()
xgboost.fit(X_train, y_train)

xgboost_prediction = xgboost.predict(X_test)
xgboost_accuracy = accuracy_score(y_test, xgboost_prediction)
xgboost_precision = precision_score(
    y_test, xgboost_prediction, average="weighted")
xgboost_recall = recall_score(y_test, xgboost_prediction, average="weighted")
xgboost_f1_score = f1_score(y_test, xgboost_prediction, average="weighted")


print("Prediciton:    ", xgboost_prediction)
print('_' * 75)

print("Accuracy:" + "\t" + f"{(xgboost_accuracy * 100)}%")
print("Precision:" + "\t" + f"{(xgboost_precision * 100)}%")
```

```
print("Recall:" + "\t\t" f"{(xgboost_recall * 100)}%")
print("F1-Score:" + "\t" + f"{(xgboost_f1_score * 100)}%")
```

```
_____Evaluation for XGBoost_____
Prediciton:     [6 6 6 ... 6 4 4]

________________________________________________________________________________
Accuracy:       99.8640380693406%
Precision:      99.84879150685241%
Recall:         99.8640380693406%
F1-Score:       99.8511626833975%
```

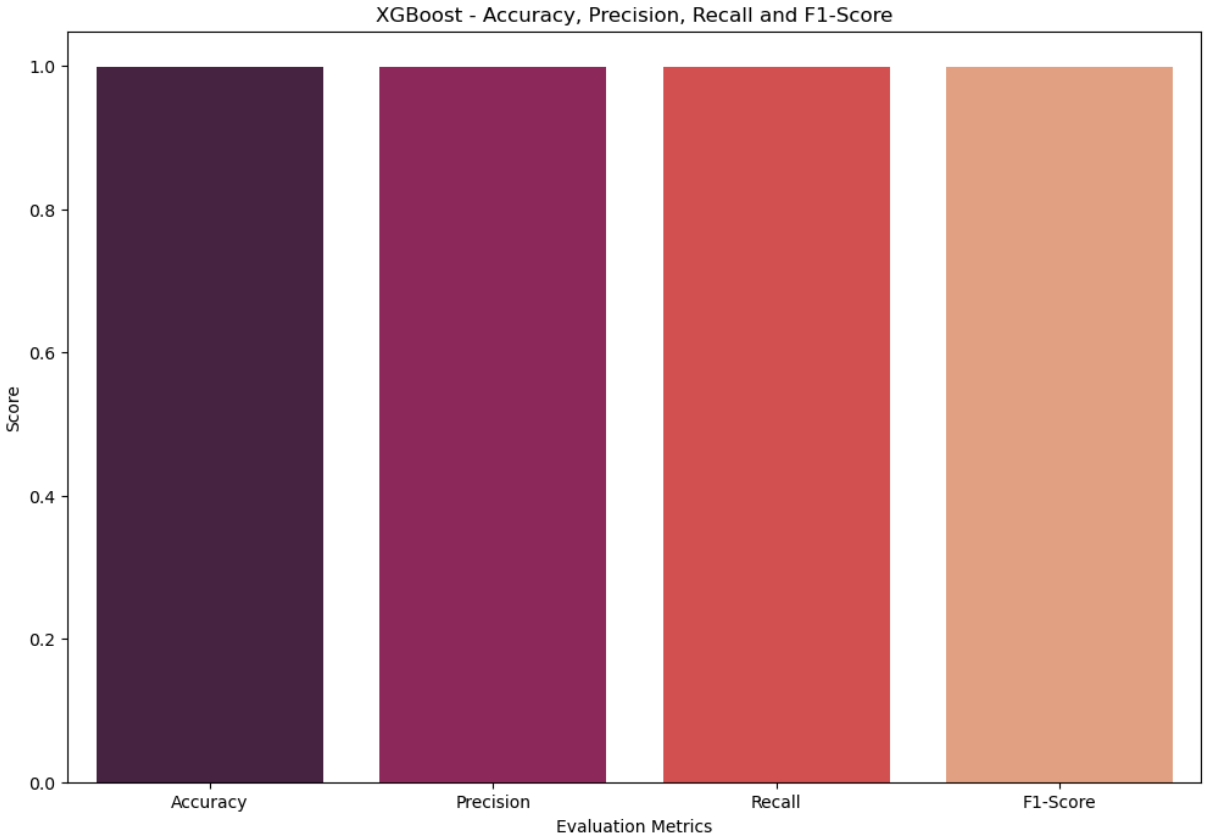
In [97]:
```
print("Confusion Matrix For XGBoost - 'Labels Test' & 'Prediction':")
print('-' * 93)
print(confusion_matrix(y_test, xgboost_prediction))
```

```
Confusion Matrix For XGBoost - 'Labels Test' & 'Prediction':
---------------------------------------------------------------------------------
---------
[[  25    0    0    0    0    0    0    0    0    0    0    0    0]
 [   0    1    0    0    0    0    0    0    0    0    0    0    0]
 [   0    0    1    0    0    0    0    0    0    0    0    0    0]
 [   0    0    0   19    0    0    0    0    0    0    0    0    0]
 [   0    0    0    0 1412    0    0    0    0    0    0    0    0]
 [   0    0    0    1    0    0    0    0    0    0    0    0    0]
 [   0    1    0    0    0    0 1381    0    0    0    0    0    0]
 [   0    0    0    0    0    0    0    4    0    0    0    0    0]
 [   0    0    0    0    0    0    0    0   11    0    0    0    0]
 [   0    0    0    0    0    0    0    0    0   19    0    0    0]
 [   0    0    0    0    0    0    0    0    0    0   33    0    0]
 [   0    0    0    0    0    0    0    0    0    0    0   16    0]
 [   0    0    0    0    0    0    2    0    0    0    0    0   16]]
```

In [98]:
```
print("Classification Report For XGBoost - 'Labels Test' & 'Prediction':")
print("-" * 97)
print(classification_report(y_test, xgboost_prediction))
```

```
Classification Report For XGBoost - 'Labels Test' & 'Prediction':
--------------------------------------------------------------------------------
-------------
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        25
           1       0.50      1.00      0.67         1
           2       1.00      1.00      1.00         1
           3       0.95      1.00      0.97        19
           4       1.00      1.00      1.00      1412
           5       0.00      0.00      0.00         1
           6       1.00      1.00      1.00      1382
           7       1.00      1.00      1.00         4
           8       1.00      1.00      1.00        11
           9       1.00      1.00      1.00        19
          10       1.00      1.00      1.00        33
          11       1.00      1.00      1.00        16
          12       1.00      0.89      0.94        18

    accuracy                           1.00      2942
   macro avg       0.88      0.91      0.89      2942
weighted avg       1.00      1.00      1.00      2942
```

In [99]:
```
season = ["Accuracy", "Precision", "Recall", "F1-Score"]
season_score = [xgboost_accuracy, xgboost_precision, xgboost_recall, xgboost_f1_sco

sns.barplot(x=season, y=season_score, palette="rocket")
plt.title("XGBoost - Accuracy, Precision, Recall and F1-Score")
plt.xlabel("Evaluation Metrics")
plt.ylabel("Score")
plt.show()
```

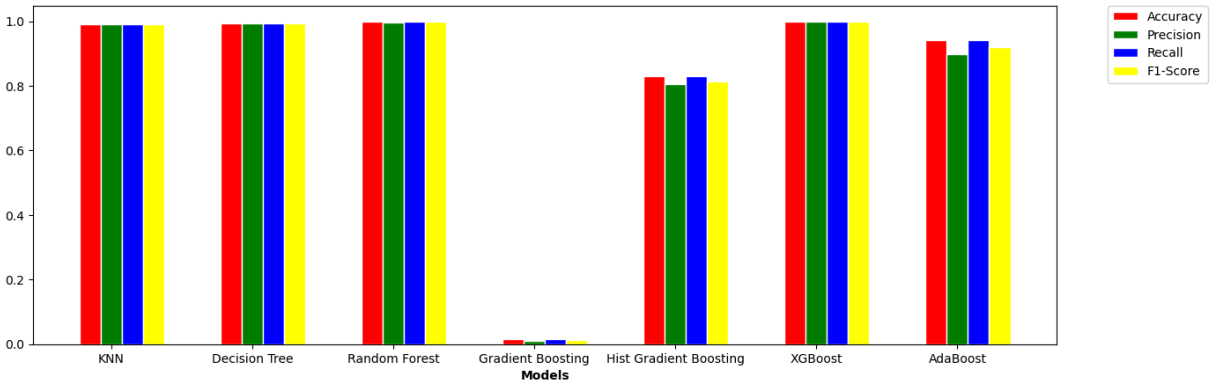


Comparison

In [108…

```
import matplotlib.pyplot as plt
import numpy as np

models = [
    "KNN",
    "Decision Tree",
    "Random Forest",
    "Gradient Boosting",
    "Hist Gradient Boosting",
    "XGBoost",
    "AdaBoost",
]
accuracy = [
    k_nearest_neighbors_accuracy,
    decision_tree_accuracy,
    random_forest_classifier_accuracy,
    gradient_boosting_accuracy,
    hist_gradient_boosting_accuracy,
    xgboost_accuracy,
    ada_boost_accuracy,
]
precision = [
    k_nearest_neighbors_precision,
    decision_tree_precision,
    random_forest_classifier_precision,
    gradient_boosting_precision,
```

```
    hist_gradient_boosting_precision,
    xgboost_precision,
    ada_boost_precision,
]
recall = [
    k_nearest_neighbors_recall,
    decision_tree_recall,
    random_forest_classifier_recall,
    gradient_boosting_recall,
    hist_gradient_boosting_recall,
    xgboost_recall,
    ada_boost_recall,
]
f1_score = [
    k_nearest_neighbors_f1_score,
    decision_tree_f1_score,
    random_forest_classifier_f1_score,
    gradient_boosting_f1_score,
    hist_gradient_boosting_f1_score,
    xgboost_f1_score,
    ada_boost_f1_score,
]

barWidth = 0.15

r1 = np.arange(len(accuracy))
r2 = [x + barWidth for x in r1]
r3 = [x + barWidth for x in r2]
r4 = [x + barWidth for x in r3]

plt.figure(figsize=(15, 5))

plt.bar(r1, accuracy, color="red", width=barWidth, edgecolor="white", label="Accura
plt.bar(
    r2, precision, color="green", width=barWidth, edgecolor="white", label="Precisi
)
plt.bar(r3, recall, color="blue", width=barWidth, edgecolor="white", label="Recall"
plt.bar(
    r4, f1_score, color="yellow", width=barWidth, edgecolor="white", label="F1-Scor
)

plt.xlabel("Models", fontweight="bold")
plt.xticks([r + barWidth for r in range(len(accuracy))], models)

plt.legend(bbox_to_anchor=(1.05, 1), loc="upper left", borderaxespad=0.0)
plt.show()
```



Deep Learning

In [110…
```
import numpy as np

num_classes = len(np.unique(y_train))
print("Number of classes:", num_classes)
```

```
Number of classes: 13
```

In [111…
```
X_train = X_train.reshape(X_train.shape[0], X_train.shape[1], 1)
X_test = X_test.reshape(X_test.shape[0], X_test.shape[1], 1)
```

In [112…
```
print(np.unique(y_train))
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12]
```

In [113…
```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv1D, MaxPooling1D, Flatten, Dense

# Number of input features
num_features = X_train.shape[1]

# Number of output classes
num_classes = len(np.unique(y_train))

# Reshape for CNN
X_train = X_train.reshape(X_train.shape[0], X_train.shape[1], 1)
X_test = X_test.reshape(X_test.shape[0], X_test.shape[1], 1)

# Build model
model = Sequential([
    Conv1D(32, kernel_size=3, activation='relu',
           input_shape=(num_features, 1)),
    MaxPooling1D(pool_size=2),
    Flatten(),
    Dense(64, activation='relu'),
    Dense(num_classes, activation='softmax')
])

# Compile
```

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train
model.fit(X_train, y_train, epochs=10, batch_size=32)

# Evaluate
loss, accuracy = model.evaluate(X_test, y_test)
print("Accuracy:", accuracy)
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for Ten
sorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use
WSL2 or the TensorFlow-DirectML plugin.
Epoch 1/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9672 - loss: 0.2474
Epoch 2/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9889 - loss: 0.0486
Epoch 3/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9904 - loss: 0.0378
Epoch 4/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9932 - loss: 0.0292
Epoch 5/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9939 - loss: 0.0230
Epoch 6/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9946 - loss: 0.0201
Epoch 7/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9955 - loss: 0.0168
Epoch 8/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9958 - loss: 0.0160
Epoch 9/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9966 - loss: 0.0131
Epoch 10/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9966 - loss: 0.0132
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9956 - loss: 0.0248
Accuracy: 0.9955812096595764
```

In [115…

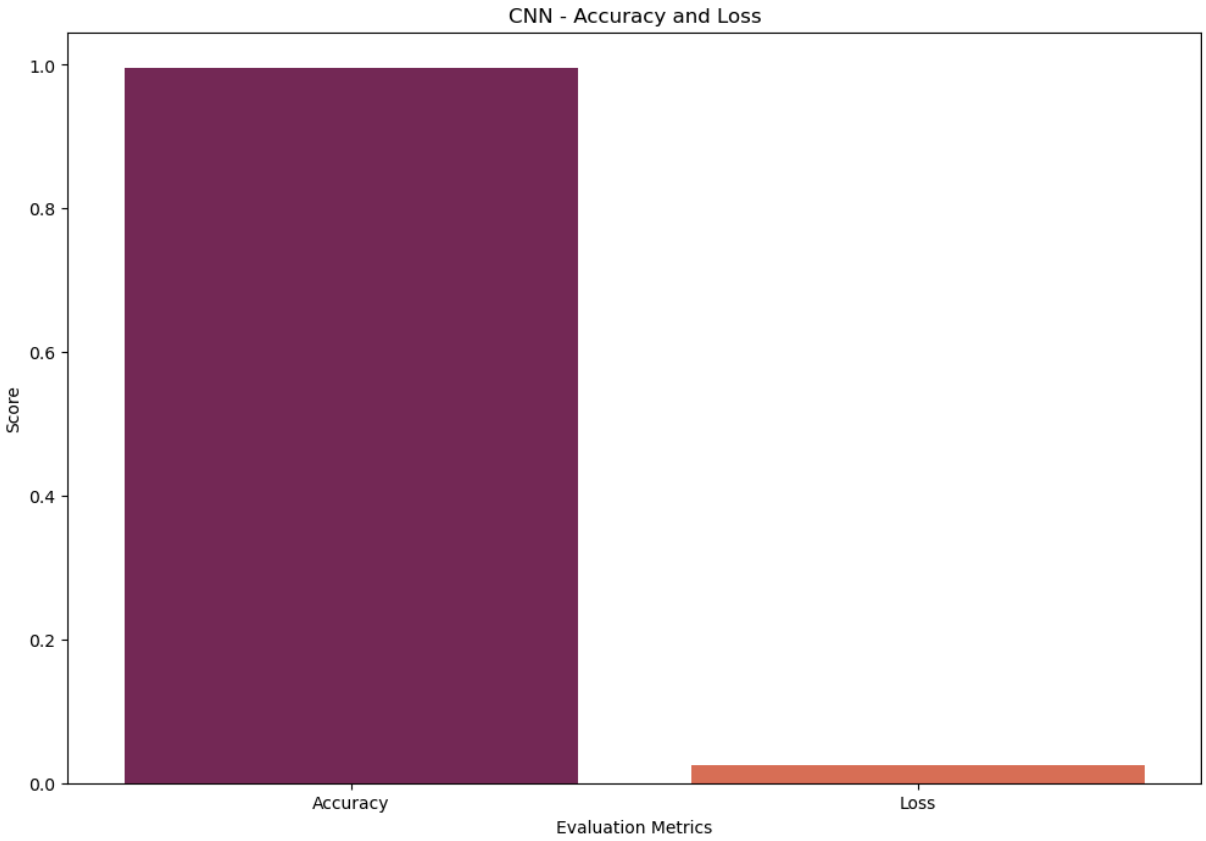
```
print("Prediction:", model)
print("_" * 75)

print(f"Accuracy:\t{accuracy*100:.2f}%")
print(f"Loss:\t\t{loss:.4f}")
```

```
Prediction: <Sequential name=sequential_1, built=True>
___________________________________________________________________________
Accuracy:       99.56%
Loss:           0.0248
```

In [118…

```
season = ["Accuracy", "Loss"]
season_score = [accuracy, loss]

sns.barplot(x=season, y=season_score, palette="rocket")
plt.title("CNN - Accuracy and Loss")
plt.xlabel("Evaluation Metrics")
```

```
plt.ylabel("Score")
plt.show()
```



Autoencoder

In [119…

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Define the Autoencoder model
model = Sequential()
model.add(Dense(128, activation="relu", input_shape=(num_features,)))
model.add(Dense(64, activation="relu"))
model.add(Dense(128, activation="relu"))
model.add(Dense(num_features, activation="linear"))

# Compile the model
model.compile(optimizer="adam", loss="mean_squared_error")

# Train the model
model.fit(X_train, X_train, epochs=10, batch_size=32)

# Use the trained model for anomaly detection
reconstructed_data = model.predict(X_test)
auto_encoder_loss = tf.keras.losses.mae(reconstructed_data, X_test)
auto_encoder_accuracy = tf.keras.metrics.binary_accuracy(reconstructed_data, X_test
```

```
Epoch 1/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - loss: 0.3094
Epoch 2/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0968
Epoch 3/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0486
Epoch 4/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.0386
Epoch 5/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.0421
Epoch 6/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.0386
Epoch 7/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.0231
Epoch 8/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0180
Epoch 9/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0228
Epoch 10/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0114
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```

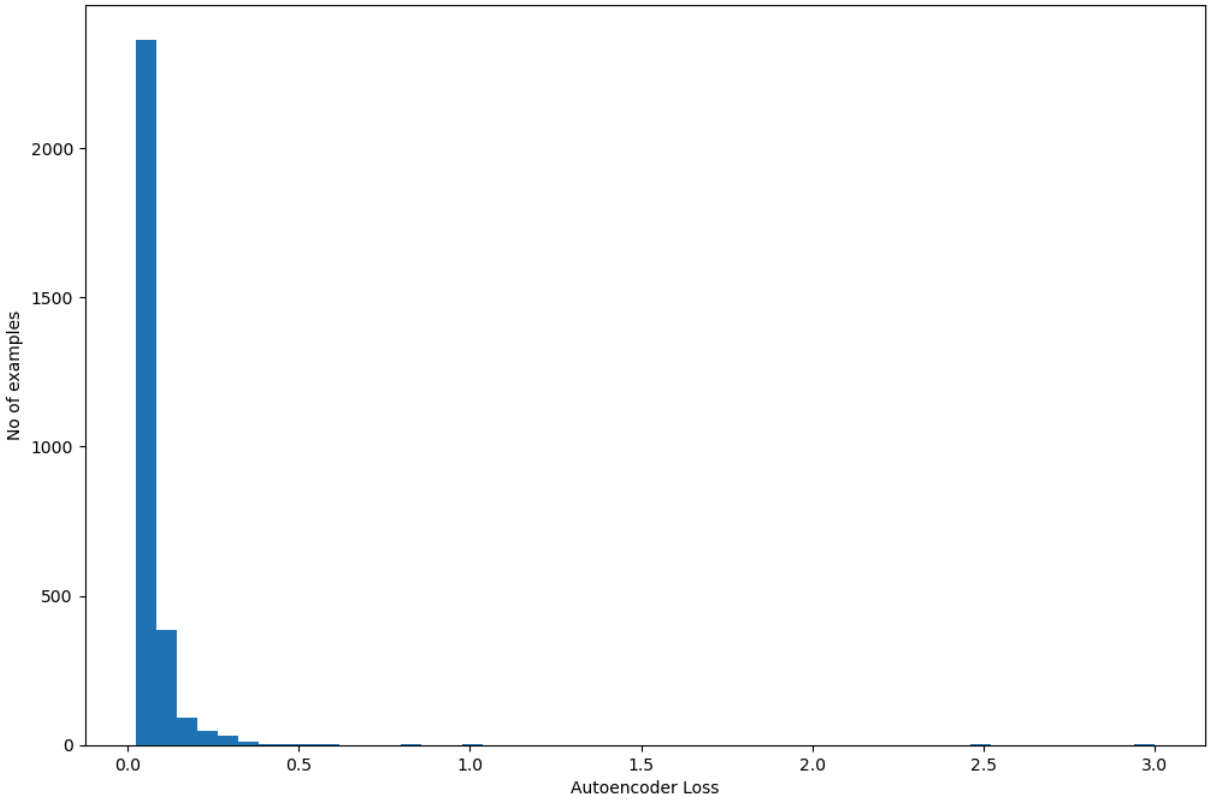
In [120…

```python
print("Prediciton:    ", model)
print('_' * 75)

print("Accuracy:" + "\t" + f"{(auto_encoder_accuracy * 100)}%")
print("Loss:" + "\t\t" + f"{(auto_encoder_loss * 100)}%")
```

```
Prediciton:     <Sequential name=sequential_2, built=True>
___________________________________________________________________________
Accuracy:       [[0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 ...
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]]%
Loss:           [ 5.05728337 10.79346112  5.75157837 ...  9.66396504  2.96880651
  2.87822754]%
```

In [121…

```python
# plot the loss distribution of the test set
plt.hist(auto_encoder_loss, bins=50)
plt.xlabel("Autoencoder Loss")
plt.ylabel("No of examples")
plt.show()
```



Gated Recurrent Unit (GRU)

In [122…

```
# Apply gru to the dataset
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GRU, Dense

# Define the GRU model
model = Sequential()
model.add(GRU(32, input_shape=(num_features, 1)))
model.add(Dense(num_classes, activation="softmax"))

# Compile the model
model.compile(
    optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"]
)

# Train the model
model.fit(X_train, y_train, epochs=10, batch_size=32)

# Evaluate the model
gru_loss, gru_accuracy = model.evaluate(X_test, y_test)
```

```
Epoch 1/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 9s 24ms/step - accuracy: 0.8375 - loss: 0.9464
Epoch 2/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - accuracy: 0.9325 - loss: 0.2773
Epoch 3/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - accuracy: 0.9491 - loss: 0.2028
Epoch 4/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 5s 25ms/step - accuracy: 0.9643 - loss: 0.1722
Epoch 5/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 5s 25ms/step - accuracy: 0.9668 - loss: 0.1481
Epoch 6/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 5s 24ms/step - accuracy: 0.9675 - loss: 0.1321
Epoch 7/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - accuracy: 0.9704 - loss: 0.1189
Epoch 8/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - accuracy: 0.9765 - loss: 0.1053
Epoch 9/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 4s 21ms/step - accuracy: 0.9787 - loss: 0.0953
Epoch 10/10
215/215 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - accuracy: 0.9786 - loss: 0.0901
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.9782 - loss: 0.0883
```

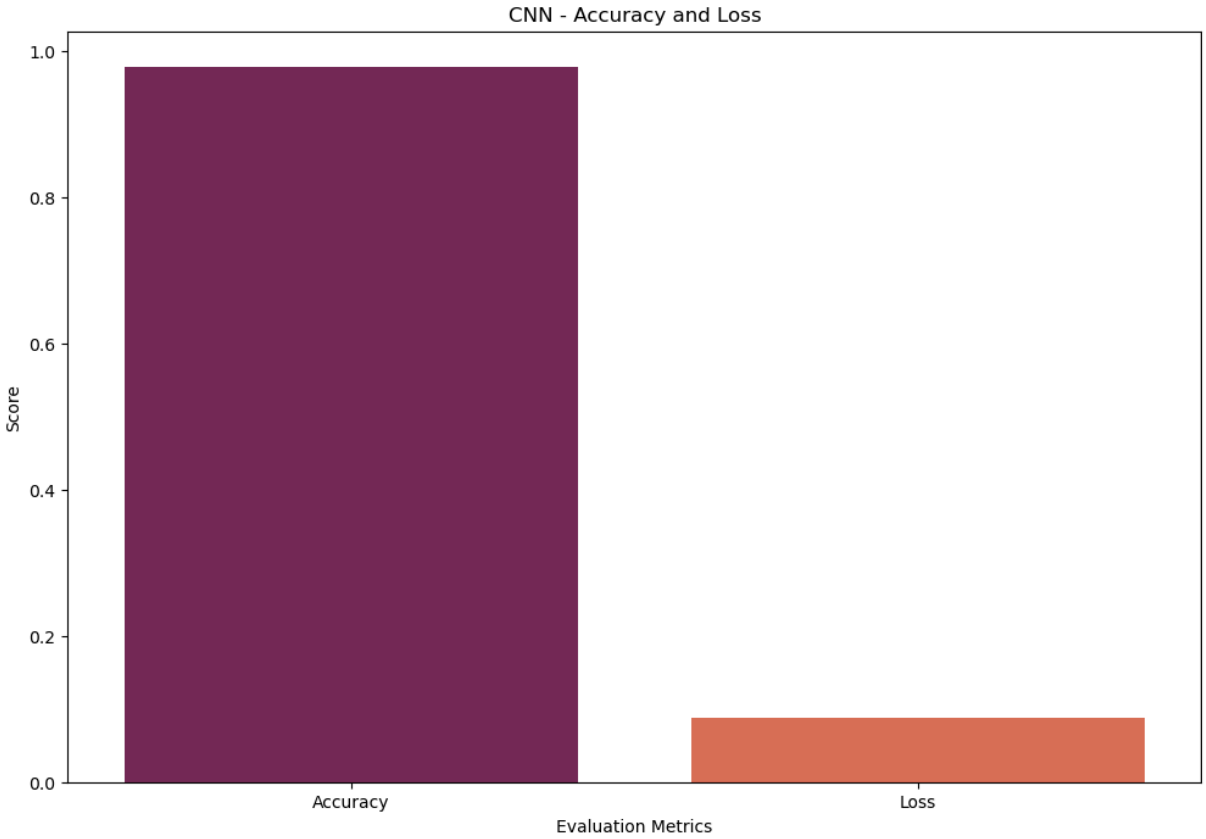
In [123…

```python
print("Prediciton:    ", model)
print('_' * 75)

print("Accuracy:" + "\t" + f"{(gru_accuracy * 100)}%")
print("Loss:" + "\t\t" + f"{(gru_loss * 100)}%")
```

```
Prediciton:     <Sequential name=sequential_3, built=True>
___________________________________________________________________________
Accuracy:       97.82460927963257%
Loss:           8.827356994152069%
```

In [124…

```python
season = ["Accuracy", "Loss"]
season_score = [gru_accuracy, gru_loss]

sns.barplot(x=season, y=season_score, palette="rocket")
plt.title("CNN - Accuracy and Loss")
plt.xlabel("Evaluation Metrics")
plt.ylabel("Score")
plt.show()
```


CNN - Accuracy and Loss
1.0
0.8
0.6
0.4
0.2
0.0
Score
Accuracy
Loss
Evaluation Metrics